

Marc Bessa Hoffmann

Efficient and Interpretable Dynamic Slicing of Neural Networks: A Block-Based Approach

March 30, 2026

supervised by:

Prof. Dr. Sibylle Schupp
Daniel Rashedi

Statutory Declaration

I herewith declare that I have composed the present thesis myself and without use of any other than the cited sources and aids. Sentences or parts of sentences quoted literally are marked as such; other references with regard to the statement and scope are indicated by full details of the publications concerned. The thesis in the same or similar form has not been submitted to any examination body and has not been published. This thesis was not yet, even in part, used in another examination or as a course performance.

Hamburg, March 30, 2026

Marc Bessa Hoffmann

Text source: Goethe University, Department of Economics

Abstract

Neural networks achieve strong performance in many domains but are often considered difficult to interpret. Understanding which internal components contribute to a specific prediction is an active research topic in explainable artificial intelligence (XAI). Program slicing has recently been adapted from traditional software systems to address this problem. NNSlicer propagates contribution scores for each neuron backward from the prediction output through the network, yielding a slice: a subset of neurons and synapses that are most relevant to a given prediction. Despite its success, NNSlicer operates at a fine-grained neuron level and is time-consuming, taking several minutes to slice even smaller ResNet models.

To improve runtime efficiency, we build on the backward contribution analysis of NNSlicer and propose a hierarchical block-based slicing approach that leverages the structural layout of residual networks. For each input, we measure the deviation of neuron responses from their dataset averages. We aggregate these deviations into channel-level and block-level indicators that express how strongly each component reacts to the input. Based on these indicators, we first filter inactive blocks, then inactive channels, and only then perform neuron-level analysis on the remaining components. For the neuron-level analysis, we propose a vectorized implementation that operates on tensors to further improve runtime.

Our vectorized implementation alone slices ResNet-18 in about two seconds compared to over 540 seconds reported for NNSlicer. Even ResNet-101, which was previously infeasible to slice in practice, can now be processed in about 7.7 seconds. Block-based slicing further increases efficiency while preserving slice quality: for ResNet-101, runtime drops from 7.7 s to 6.2 s and the slice size decreases from 41.8 % to 23.5 %. Slice quality is evaluated on a custom ResNet with 27 residual blocks and 855K parameters. Model pruning shows that a block-pruned model retains 91.9% target accuracy with only 39.7% of its parameters. Counterfactual experiments further confirm that progressively removing neurons inside the slice degrades the prediction, whereas the prediction remains stable when removing neurons outside the slice. Finally, we observe that channels and blocks in the last residual blocks exhibit interpretable patterns across slices of semantically related classes.

Contents

1	Introduction	1
2	Background	5
2.1	Neural Networks	5
2.2	Convolutional Neural Networks	9
2.3	Residual Networks	11
2.4	Program Slicing	14
3	Related Work	17
3.1	Neural Network Slicing	17
3.2	Model Interpretability	18
3.3	Program Analysis for Neural Networks	19
4	NNSlicer	21
4.1	Profiling	21
4.2	Forward Analysis	22
4.3	Backward Analysis	23
5	Block-Based Dynamic Slicing	27
5.1	Delta Aggregation	27
5.2	Block Slicing	30
5.3	Channel Slicing	34
5.4	Vectorized Backward Analysis	36
6	Evaluation	41
6.1	Experimental Setup	41
6.2	Results	42
6.3	Threats to Validity	57
7	Summary and Future Work	61
7.1	Summary	61
7.2	Future Work	62

List of Figures

2.1	A neural network with fully connected layers. An input layer with two neurons. Two hidden layers with three and two neurons. One output neuron for the prediction. The edges represent the synapses.	7
2.2	Convolution operation. A 3×3 kernel slides over a 5×5 input with stride 1 and no padding to produce a 3×3 feature map. The highlighted positions show the first computation	9
2.3	Example of a convolutional neural network. An input image passes through convolutional, pooling, and batch normalization layers before being classified by fully connected layers	10
2.4	Max pooling and average pooling for a 2×2 region. Max pooling selects the maximum, average pooling computes the mean	11
2.5	A residual block with a skip connection. The input x is added to the output of the block $F(x)$, yielding $y = F(x) + x$	12
4.1	Profiling phase. Mean activations $\bar{y}_n$ are computed for every neuron $n \in \mathcal{N}$ over the training dataset $\mathcal{I}$	22
4.2	Forward analysis phase. Activation deltas $\Delta_n = y_n(\xi) - \bar{y}_n$ are computed for a specific input $\xi = [1, 2]$	22
4.3	Backward analysis phase. Contributions are traced backward from the target neuron. Orange nodes and edges form the computed slice, gray elements are excluded	25
5.1	Hierarchical delta aggregation. Neuron deltas are aggregated into channel deltas, then into layer deltas, and finally into block deltas . .	28
5.2	Block slicing example with $\beta = 0.8$. Blocks b_1 , b_2 , and b_4 are retained. Block b_3 is skipped through its identity shortcut. Orange elements are part of the slice, gray elements are not.	32
5.3	FX graph of a residual block. The addition node marks the end of the block. The main path and shortcut type are identified by inspecting the parent nodes of the addition.	33
5.4	Channel-level slicing example for a convolutional layer with $ \mathcal{C} = 5$ channels. With $\alpha = 0.8$ and $E_{\text{ch}} = 5.0$, channels c_3 and c_1 are retained (orange) and the remaining three are excluded (gray).	35
5.5	Tensor inputs for the vectorized backward contribution of a fully connected layer with $m = 3$ output and $n = 4$ input neurons.	37
5.6	Intermediate contribution matrix after element-wise multiplication with broadcasting. Each entry represents the raw contribution value for one output-input connection.	38
5.7	After applying sign, each entry is $+1$, 0 , or -1	38

6.1	Target accuracy as a function of the threshold for channel, block, and combined pruning. Block slicing retains high accuracy even at aggressive thresholds, while channel slicing degrades earlier	49
6.2	Necessity evaluation on ResNet-CIFAR. Target probability drops as neurons inside the slice are progressively scaled down.	51
6.3	Sufficiency evaluation on ResNet-CIFAR. Target probability as neurons outside the slice are progressively scaled down.	52
6.4	Cosine similarity between slice contribution vectors for ResNet-CIFAR across blocks 19–27. Same-class pairs show the highest similarity in the final blocks, followed by semantically related pairs (cat–dog, truck–automobile). Cat–truck shows the lowest similarity throughout. . . .	54
6.5	Channels 23 and 31: the two channels with the largest overlap between cat and dog in block 27 of ResNet-CIFAR. Cat and dog show high contributions, though other animal classes also contribute notably. . .	55
6.6	Channels 50 and 4: the two channels with the largest overlap between truck and automobile in block 27 of ResNet-CIFAR. Truck and automobile show high contributions while other classes remain low. . . .	56
6.7	Channels 49 and 16: the two channels with the largest overlap across animal classes in block 27 of ResNet-CIFAR. All six animal classes show high contributions while vehicle classes remain low.	56
6.8	Channels 11 and 28: the two channels with the largest overlap across vehicle classes in block 27 of ResNet-CIFAR. All four vehicle classes show high contributions while animal classes remain low.	57

List of Tables

2.1	ResNet variants with their block types and configurations [8]. Shallower models use BasicBlocks, deeper models use BottleneckBlocks . . .	13
4.1	Backward contribution rules for each operator type. $CONTRIB_n$ is the cumulative contribution of neuron n . Δy , Δx_i are the activation deltas of the output and i -th input neuron, respectively [6]	24
6.1	Models used in the evaluation with their parameter counts and residual blocks. ImageNet models use pretrained torchvision weights. ResNet-CIFAR is trained on CIFAR-10	43
6.2	Backward analysis time per slice without channel or block filtering (baseline). The upper section isolates the effect of vectorization on CIFAR-10 ResNet. The lower section shows ImageNet ResNet models with our vectorized implementation.	44
6.3	Effect of block slicing on runtime and slice size for three β thresholds. Skipped blocks are excluded from backward analysis entirely	44
6.4	Effect of channel slicing on runtime and slice size for three α thresholds. Lower α retains fewer channels, reducing slice size only	45
6.5	Combined channel and block slicing with $\alpha = 0.85$ and $\beta = 0.80$, showing the cumulative effect of both slicing strategies	46
6.6	Channel pruning results for varying α . Target and non-target accuracy after zeroing out channels excluded by the slice	47
6.7	Block pruning results for varying β . Target and non-target accuracy after zeroing out neurons in blocks excluded by the slice	48
6.8	Combined pruning results with fixed $\alpha = 0.85$ and varying β , showing how block filtering complements channel filtering	49

Listings

2.1	Example program P with slicing criterion $C = (8, \{\mathbf{z}\})$	14
2.2	Static slice $P_C = \{1, 2, 4, 5, 6, 7, 8\}$ for $C = (8, \{\mathbf{z}\})$	15
2.3	Dynamic slice $P_C = \{1, 2, 4, 5, 8\}$ for $C = (8, \{\mathbf{z}\})$ with input $x = 3$. .	15

List of Algorithms

1	ComputeContrib [6]: computing the contributions of neurons and synapses for an input sample	24
2	BlockSlice: identifying residual blocks to skip	31
3	ChannelSlice: identifying channels to skip	34
4	Loop-based backward contribution for a fully connected layer	37

1 Introduction

Neural networks are widely used in domains such as medical imaging, autonomous driving, and natural language processing [1]. Despite their strong performance, they are often treated as black boxes whose internal decision process is difficult to understand. In safety-critical settings, the lack of transparency makes it hard to assess the reliability of individual predictions and highlights the need for trustworthy models. This need has motivated the field of explainable artificial intelligence (XAI), which aims to make model decisions more understandable to humans [2].

In recent years, program slicing has emerged as an approach within XAI that focuses on internal components of neural networks. Program slicing is a technique that was originally developed for traditional software systems. The core idea is to identify the parts of a program that are relevant to a particular computation or output [3]. Over several decades, it has been successfully applied to tasks such as debugging, program comprehension, and testing [4]. The concept of program slicing has been adapted to neural networks to analyze which internal components contribute most to a specific prediction [5, 6, 7].

Among these, NNSlicer is a dynamic program slicing approach that operates in three phases [6]. First, a profiling phase runs the training data through the network and computes the mean activation at each neuron. In the second phase, forward analysis compares the activations for a specific input against these means. The resulting delta for each neuron indicates how much its activation deviates from the average. Finally, the backward analysis traces contributions from the output back to identify the relevant neurons and synapses. The output of NNSlicer is a slice of a neural network that identifies the most relevant neurons and synapses with positive or negative contributions. NNSlicer shows the utility of such slices for applications like adversarial input detection, targeted model pruning, and selective model protection.

Although NNSlicer demonstrates that program slicing can be applied to neural networks, it operates on a fine-granular neuron level. While this provides precise per-neuron contributions, modern architectures can contain millions of parameters distributed across hundreds of layers. Slicing such models at the neuron level is time-consuming, taking several minutes even for smaller models and becoming infeasible for larger ones. Making slicing more efficient in terms of runtime is therefore our primary objective.

In this thesis, we build on NNSlicer and propose a hierarchical block-based slicing approach that leverages the structural layout of the neural network. We focus exclusively on convolutional residual networks (ResNets), as they provide a clear residual-block structure for higher-level slicing. Our method first filters at the residual block level, then at the channel level, and finally performs neuron-level analysis on the remaining components. To determine which blocks and channels to retain, we extend the forward analysis of NNSlicer to aggregate neuron deltas not only per neuron, but also per channel and per residual block. These aggregated deltas indicate how strongly each component reacts to a given input compared to its average

behavior. Based on these aggregated deltas, we rank blocks and channels in descending order and retain only those that account for a configurable fraction of the total delta. Blocks and channels below this threshold are filtered out and excluded from the slice. For the blocks and channels that are included in the slice, we compute the neuron-level contributions within each block and channel as NNSlicer does.

Since the original NNSlicer iterates over individual neurons in Python loops, the computation time grows with the number of neurons and does not scale to larger models. As a second contribution alongside block-based slicing, we propose a vectorized PyTorch reimplementation that computes neuron contributions through tensor operations instead of processing them one by one. Together, block-based slicing and the vectorized implementation aim to increase runtime efficiency and make neural network slicing practical for larger models.

Our evaluation is structured around three research questions. The first research question focuses on runtime performance and slice size. To assess slice size, we measure the proportion of active neurons and synapses within a slice compared to the entire network. We evaluate our method on ResNet-18, ResNet-34, and ResNet-101 to cover models of increasing depth and complexity. Our vectorized reimplementation slices a ResNet-18 in approximately two seconds, while NNSlicer reports over 540 seconds for the same model. Even larger models such as ResNet-101 with 44.5 million parameters, previously infeasible to slice with NNSlicer, can now be sliced in about 7.7 seconds with our vectorized implementation. Block-based slicing further reduces both runtime and slice size. For a ResNet-101 slice, we achieve a runtime improvement of 19 % by dropping from 7.67 s to 6.20 s. Block-based slicing is particularly effective at producing smaller slices. For the same ResNet-101 slice, the slice size reduces from 41.8 % without block-based slicing to 23.5 % with block-based slicing.

Since efficiency alone is not sufficient, the second research question examines whether block-based slicing preserves slice quality. By quality, we mean whether the slice still captures the neurons and synapses that are most relevant to the prediction. The evaluation of slice quality is conducted on a custom ResNet that has been trained on the CIFAR-10 dataset and consists of 27 residual blocks with 855K parameters. The architecture follows the CIFAR-10 ResNet design described by He et al. [8], with three groups of nine BasicBlocks each and only identity shortcuts. We chose a model with 27 residual blocks to provide sufficient depth for evaluating the block-based slicing approach. CIFAR-10 was chosen because it is a widely used benchmark that was also used in the original NNSlicer evaluation. It can be automatically downloaded with PyTorch and makes our results fully reproducible.

We use two different experiments to assess quality. In model pruning, we remove all components outside the slice and measure whether the pruned model still correctly classifies the target class. Block-based slicing retains 91.9 % target accuracy while reducing the model to 39.7 % of its original size. As a second experiment, we use counterfactual reasoning, which asks what would happen if parts of the network were different. We examine two aspects: necessity tests whether removing neurons inside the slice degrades the prediction, and sufficiency tests whether the prediction remains stable when neurons outside the slice are removed. Block-based slicing yields

counterfactual behavior nearly identical to slicing without any filtering while producing smaller slices. In contrast, aggressive block-based slicing shows a rapid drop in prediction confidence, confirming that the filtered components are indeed relevant.

The third research question investigates interpretability at the block and channel level. As an exploratory step, we examine whether patterns can be observed between slices of different classes. We compare classes where similarities are expected: automobile and truck as vehicles, and cat and dog as animals. On the block level, we observe overlaps in the final blocks of the network. When zooming into the last block, we find channels with strongly shared contributions between cat and dog as well as between automobile and truck.

Overall, our results show that block-based slicing and our vectorized reimplementations of NNSlicer substantially reduce runtime and slice size while preserving slice quality. We also find that the resulting slices support interpretability by highlighting class-specific patterns in the last residual blocks and their most influential channels.

In summary, the contributions of this thesis are as follows:

- Block-based slicing approach: a hierarchical extension that filters at the residual block and channel level before performing neuron-level analysis.
- Vectorized PyTorch implementation of NNSlicer that replaces neuron-by-neuron loops with tensor operations.
- Evaluation of runtime and slice size across ResNet models of increasing depth. Slice quality and interpretability are evaluated on a custom ResNet model using model pruning and counterfactual reasoning.

We structure this thesis as follows. First, we introduce neural networks and the architectures used in this work, namely convolutional and residual networks. We then describe program slicing and how the concept can be applied to neural networks. Next, we review related work. Since our approach builds directly on NNSlicer, we dedicate a separate chapter to its pipeline. We then introduce our block-based slicing approach and the vectorized implementation. Finally, we present the evaluation results and conclude with a summary of findings and directions for future work.

2 Background

In this chapter, we provide the technical background and definitions used throughout this work. We start by introducing the concept of a neuron and how neurons are combined into a neural network. After that, we describe two neural network architectures that are of particular interest: convolutional neural networks and residual networks. Finally, we discuss program slicing and how it can be applied to neural networks.

2.1 Neural Networks

Neural networks are mathematical models that draw inspiration from biological neural systems. In the brain, a neuron fires when the sum of its incoming signals exceeds a threshold. The same principle underlies the mathematical neuron, where a linear combination is calculated for a set of numerical inputs to produce an output [9]. If we consider a neuron with d inputs, the input vector is defined as:

$$\mathbf{x} = (x_1, x_2, \dots, x_d)^\top \quad (2.1)$$

Each input x_i has an associated weight w_i :

$$\mathbf{w} = (w_1, w_2, \dots, w_d)^\top \quad (2.2)$$

The neuron computes a weighted sum of all inputs and adds a bias term b :

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b \quad (2.3)$$

The weights determine how much influence each input has on the output. For example, a neuron that decides whether to bring an umbrella might assign a high weight to the input *is it raining* and a low weight to *is it sunny*. While the weights control the influence of each input, the bias b shifts the activation threshold independently of the inputs. If we consider the umbrella example again, a negative bias would mean that the evidence for rain must be strong before the neuron decides to bring one. A positive bias makes the neuron more cautious, activating even with a weak signal for *is it raining*.

As defined above, z is so far only a linear combination of inputs and can only represent linear transformations. To solve more complex tasks, non-linearity is introduced by passing z through an *activation function* σ . The output of the neuron is therefore:

$$y = \sigma(z) \quad (2.4)$$

Many different activation functions exist for different purposes. For instance, the sigmoid activation function maps values between the range of 0 and 1. The output is

therefore bounded to a fixed range, which prevents activations from growing across layers. However, such functions can make training in deep networks difficult. Since the output is bounded to a small range, the gradients during backpropagation can become very small. As these small gradients are multiplied across many layers, they shrink further and can effectively vanish [10]. Rectified linear functions address this problem and have become the standard choice in modern architectures [11]. A survey covering further activation functions is provided by Dubey et al. [12].

In this work, all models that have been evaluated use the ReLU activation function (Rectified Linear Unit) [13]:

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z, & \text{if } z \geq 0 \\ 0, & \text{otherwise} \end{cases} \quad (2.5)$$

ReLU is applied to the raw output z of the linear combination of a neuron: values where $z \geq 0$ are passed through for further processing, while negative values are set to zero. Besides ReLU, another important activation function is Softmax [10]:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (2.6)$$

The Softmax activation function is used for the prediction in the final output layer. It takes the raw values of all output neurons and normalizes them into a probability distribution. For example, given three output neurons with raw values (2.0, 1.0, 0.1), Softmax produces approximately (0.66, 0.24, 0.10), indicating a 66% probability for the first class.

With activation functions in hand, we can now formalize the definitions used in this work. The following definitions for neural networks are based on the concepts introduced by Goodfellow et al. [10] and have been adapted for this work. The notation is aligned with NNSlicer, as our work builds directly on their formulation [6].

Definition 2.1 (Neuron). *A neuron n takes an input vector $\mathbf{x} \in \mathbb{R}^d$ and a weight vector $\mathbf{w} \in \mathbb{R}^d$ and computes the output $y = \sigma(z)$, where $z = \mathbf{w}^\top \mathbf{x} + b$ is the weighted sum with bias $b \in \mathbb{R}$, and σ is a non-linear activation function. We denote the set of all neurons in a network as $\mathcal{N}$.*

In practice, a single neuron is not used in isolation. Multiple neurons are connected to form a neural network, where the output of one neuron serves as input to another. These connections carry the weights introduced above and are called *synapses*.

Definition 2.2 (Synapse). *A synapse $s = (n_i, n_j, w)$ is a directed weighted connection from neuron n_i to neuron n_j with weight $w \in \mathbb{R}$. The value transmitted through the synapse is the activation y_i of the source neuron n_i . We denote the set of all synapses in a network as $\mathcal{S}$.*

Neurons and synapses in a network are organized into layers. The first layer is the input layer, which receives the raw input data. The last layer is the output

layer, which produces the final prediction. Layers between input and output are called hidden layers, as their activations are internal to the network and not directly observable.

Definition 2.3 (Layer). *A layer $l = (\mathcal{N}_l, \mathcal{S}_l)$ consists of a subset of neurons $\mathcal{N}_l \subset \mathcal{N}$ and a subset of synapses $\mathcal{S}_l \subset \mathcal{S}$. The synapses $\mathcal{S}_l$ are all synapses whose target neuron belongs to $\mathcal{N}_l$. We denote the set of all layers in a network as $\mathcal{L}$.*

Layers differ in how their neurons are connected to the preceding layer. In a fully connected layer, every neuron receives input from all neurons in the preceding layer.

Definition 2.4 (Fully Connected Layer). *A fully connected layer $l = (\mathcal{N}_l, \mathcal{S}_l)$ is a layer in which every neuron in $\mathcal{N}_l$ is connected to every neuron in the preceding layer $\mathcal{N}_{l-1}$, i.e., for every pair (n_i, n_j) with $n_i \in \mathcal{N}_{l-1}$ and $n_j \in \mathcal{N}_l$, there exists a synapse $(n_i, n_j, w) \in \mathcal{S}_l$.*

Figure 2.1 shows an example of a simple neural network with fully connected layers. It consists of $|\mathcal{L}| = 4$ layers, $|\mathcal{N}| = 8$ neurons, and $|\mathcal{S}| = 14$ synapses. The input layer contains two neurons (blue), followed by two hidden layers with 3 and 2 neurons (yellow). The output layer has one neuron (green). The gray edges represent the synapses.

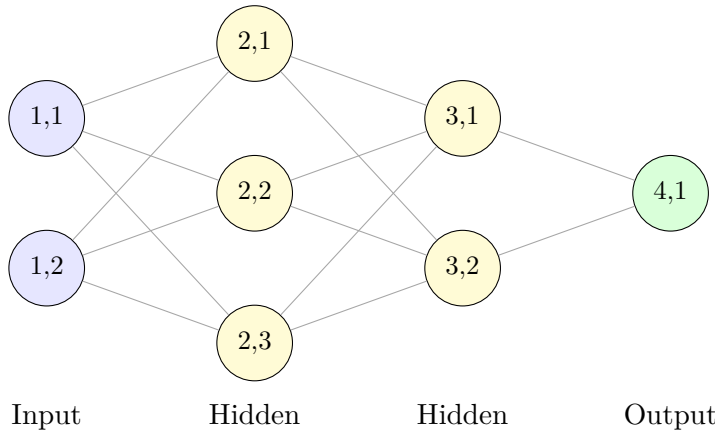


Figure 2.1: A neural network with fully connected layers. An input layer with two neurons. Two hidden layers with three and two neurons. One output neuron for the prediction. The edges represent the synapses.

More formally, a neural network can be described by its layers, neurons, and the synapses connecting them.

Definition 2.5 (Neural Network). *A neural network is a tuple $\mathcal{M} = (\mathcal{N}, \mathcal{S})$, where $\mathcal{N}$ is the set of neurons and $\mathcal{S} \subseteq \mathcal{N} \times \mathcal{N} \times \mathbb{R}$ is the set of synapses. The neurons are organized into layers $\mathcal{L} = \{l_1, \dots, l_L\}$: an input layer, one or more hidden layers, and an output layer.*

The input to a neural network $\mathcal{M}$ is a sample ξ , a numerical vector representing a single data point. We define a dataset as a collection of such samples $\mathcal{I} = \{\xi_1, \xi_2, \dots, \xi_m\}$.

The weights of a neural network must be trained before the model can be used for predictions. The following paragraph gives an overview of the training process.

Training. Initially, the weights of a network are set to small random values. The network learns by adjusting its weights and biases to minimize a loss function that measures the difference between the predicted output and the expected output. For classification tasks, a common choice is the cross-entropy loss. Given a true class label y and the predicted probability p_y from the Softmax output, it is defined as:

$$L = -\log(p_y) \quad (2.7)$$

The loss is small when the predicted probability for the correct class is high, and large when it is low. To minimize the loss, the network needs to determine how much each weight contributes to the error. This is computed using backpropagation, which applies the chain rule of calculus to propagate the gradient of the loss backwards through the network, layer by layer [14]. For a loss L and a weight w in layer l , the chain rule decomposes the gradient as:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z_l} \cdot \frac{\partial z_l}{\partial w} \quad (2.8)$$

where z_l is the output of layer l . Once the gradients are computed, the weights are updated using gradient descent. In each step, a weight w is updated by subtracting the gradient scaled by a learning rate η :

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w} \quad (2.9)$$

The learning rate η controls how large each update step is. A value too large can cause the training to overshoot, while a value too small makes training unnecessarily slow. Several optimizers have been developed to improve gradient descent, such as SGD with momentum or Adam [15, 16]. During training with such an optimizer, the network processes the dataset over multiple epochs. One epoch means every sample $\xi \in \mathcal{I}$ has been processed once. In practice, $\mathcal{I}$ is split into smaller batches, and the weights are updated after each batch.

Once the training is complete, the network can make predictions by passing new inputs through the network in a single forward pass. The slicing approach presented in this work operates on this forward pass, using the intermediate activation values to determine which parts of the network are relevant for a given prediction.

The neural networks described so far use fully connected layers, where each neuron in one layer is connected to every neuron in the next. The following section introduces convolutional neural networks, an alternative architecture specialized for processing images.

2.2 Convolutional Neural Networks

Convolutional neural networks (CNNs) are a class of neural networks designed for processing structured data such as images [17]. The core idea is the convolution operation: a small matrix called a filter (or kernel) with weights $\mathbf{w}$ and bias b slides over the input and computes a weighted sum at each position. Each filter is designed to detect a specific pattern in the input. For example, a filter can detect vertical edges by responding strongly wherever the pixel value changes from left to right. The convolution operation is illustrated in Figure 2.2.

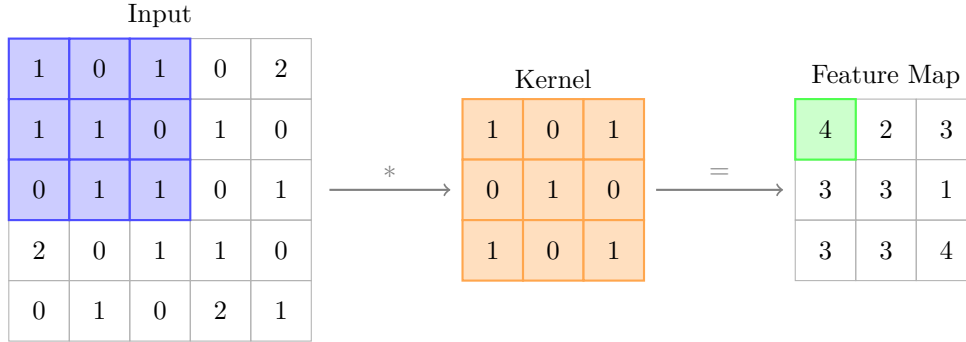


Figure 2.2: Convolution operation. A 3×3 kernel slides over a 5×5 input with stride 1 and no padding to produce a 3×3 feature map. The highlighted positions show the first computation

Stride and padding are two parameters that control the convolution and determine the dimensions of the resulting feature map. The stride defines how far the filter moves between positions. Padding adds zeros around the input borders, which allows the output to preserve the spatial dimensions of the input. In the example above, a stride of 1 was used to move the filter by one position. No padding is used. The resulting feature map is 3×3 and is also known as a channel. The terms *feature map* and *channel* are often used interchangeably. A feature map refers to the spatial output of a convolution, while a channel refers to the structural unit within a layer.

Similar to the neural network definitions, the following definitions are based on Goodfellow et al. [10] and have been adapted for this work.

Definition 2.6 (Channel). *A channel c is produced by applying a single filter with kernel weights $\mathbf{w} \in \mathbb{R}^{k \times k}$ and bias $b \in \mathbb{R}$ to an input $\mathbf{x}$. It consists of a set of neurons $\mathcal{N}_{ch}$ arranged in a spatial grid of height H and width W , connected by synapses $\mathcal{S}_{ch}$. Each neuron $n_{i,j}$ computes a linear combination of a local $k \times k$ region of the input $\mathbf{x}$ at position (i, j) .*

The feature map in Figure 2.2 has $H = W = 3$, so the channel contains 9 neurons in $\mathcal{N}_{ch}$. Each neuron $n_{i,j}$ is connected to a 3×3 input region through 9 synapses in $\mathcal{S}_{ch}$, resulting in $9 \times 9 = 81$ synapses in total. All 9 neurons share the same kernel weights $\mathbf{w}$ and bias b . Multiple filters are combined into a convolutional layer, where each filter produces one channel.

Definition 2.7 (Convolutional Layer). *A convolutional layer l consists of a set of channels $\mathcal{C}$, where each channel $c \in \mathcal{C}$ is produced by a distinct filter. The neurons of the layer are the union of all channel neurons $\mathcal{N}_l = \bigcup_{c \in \mathcal{C}} \mathcal{N}_{ch}$, and similarly for the synapses $\mathcal{S}_l = \bigcup_{c \in \mathcal{C}} \mathcal{S}_{ch}$.*

The convolutional layer in Figure 2.3 has $|\mathcal{C}| = 3$ channels. Each channel contains $30 \times 30 = 900$ neurons. The layer therefore has $|\mathcal{N}_l| = 3 \times 900 = 2,700$ neurons in total. With a 3×3 kernel, each neuron has 9 synapses, giving $|\mathcal{S}_l| = 2,700 \times 9 = 24,300$ synapses for the entire layer.

Convolutional layers are typically combined with other layer types into a full architecture. Figure 2.3 shows an example: the input image is first passed to a convolutional layer that produces three channels. These are then reduced in size by a pooling layer. A batch normalization layer normalizes the values before they are flattened and passed through fully connected layers to produce the final classification.

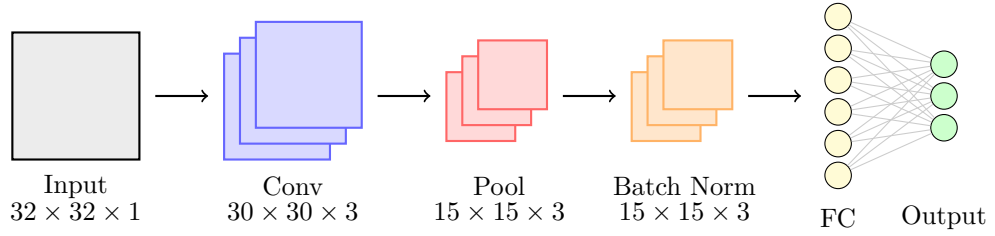


Figure 2.3: Example of a convolutional neural network. An input image passes through convolutional, pooling, and batch normalization layers before being classified by fully connected layers

Definition 2.8 (Convolutional Neural Network). *A convolutional neural network is a neural network $\mathcal{M} = (\mathcal{N}, \mathcal{S})$ with layers $\mathcal{L} = \{l_1, \dots, l_L\}$, where at least one layer $l \in \mathcal{L}$ is a convolutional layer. The remaining layers typically include pooling, batch normalization, and fully connected layers.*

The following paragraphs describe layer types that exist in convolutional neural networks.

Pooling Layers. Similar to the convolution operation, pooling slides a window over the feature map. Instead of a filter, it applies a fixed operation at each position. This reduces the spatial dimensions of the feature map, lowering the computational cost for subsequent layers. We consider two pooling operations: *max pooling* and *average pooling*. Max pooling takes the values of the region as input and selects the maximum. Average pooling computes the mean instead. While max pooling preserves the strongest activation in each region, average pooling provides a smoother summary by considering all values equally. Both pooling operations are shown in Figure 2.4 for a 2×2 region.

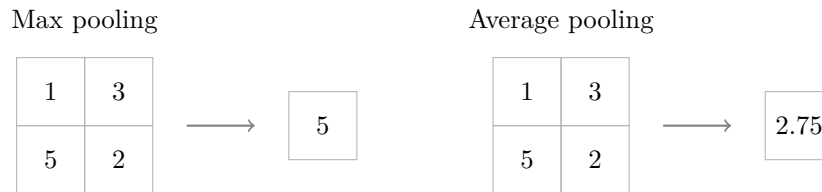


Figure 2.4: Max pooling and average pooling for a 2×2 region. Max pooling selects the maximum, average pooling computes the mean

Batch Normalization. As a network grows deeper, the activation values passed between layers can shift during training, making optimization difficult. Batch normalization addresses this by normalizing these values across a mini-batch of training samples [18].

For a given value x , the batch mean μ is subtracted and the result is divided by the standard deviation σ , centering the values around zero with unit variance:

$$\hat{x} = \frac{x - \mu}{\sigma} \quad (2.10)$$

This stabilizes training and reduces sensitivity to weight initialization. It also allows higher learning rates, which leads to faster convergence [18].

Flatten. The output of the convolutional and pooling stages consists of multiple channels c . Fully connected layers, however, expect a one-dimensional input vector. A flatten layer combines all values into a single vector so that it can be passed through fully connected layers to produce the final output predictions.

Now that the building blocks of convolutional neural networks have been introduced, we turn to a specific CNN architecture that all models in this work are based on: residual networks.

2.3 Residual Networks

Adding more layers to a neural network does not always improve performance. In theory, a deeper network should do at least as well as a shallower one, since the extra layers could just learn the identity function and pass the input through unchanged. In practice, however, standard networks struggle to learn this identity mapping [8]. Learning the identity requires all weights to reproduce the input exactly, which is difficult to achieve for every possible input. He et al. proposed residual networks (ResNets) to address this [8].

The key idea is to add skip connections that bypass one or more layers and add the original input x directly to the output. Let $H(x)$ denote the desired output of a group of layers. In a ResNet, the layers do not learn $H(x)$ directly. Instead, they learn the residual function, which is the difference between the desired output and

the input:

$$F(x) = H(x) - x \quad (2.11)$$

This can be rearranged into the core equation of a residual block, where $F(x)$ is computed by the main path and x is passed through the skip connection:

$$H(x) = F(x) + x \quad (2.12)$$

Assuming the optimal mapping is the identity, i.e., $H(x) = x$, this would mean that the result of the main path $F(x)$ is zero. Pushing all weights towards zero is much easier than learning the identity mapping $H(x) = x$ from scratch. The main path and the skip connection together form a residual block. The structure of a single residual block is shown in Figure 2.5.

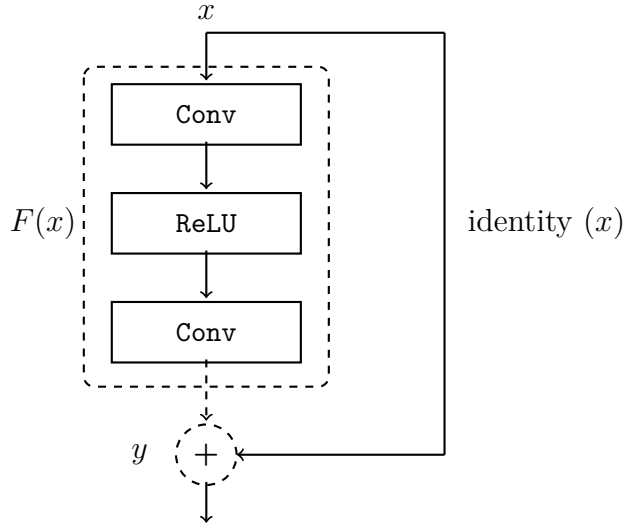


Figure 2.5: A residual block with a skip connection. The input x is added to the output of the block $F(x)$, yielding $y = F(x) + x$

The figure shows a simplified residual block with two convolutional layers and a ReLU activation in between. The left side represents the *main path*, which computes the residual function $F(x)$. The right side is the *skip connection*, which passes the input x unchanged. Both paths are combined at the addition node, producing the output $y = F(x) + x$, which corresponds to our desired mapping $H(x)$. In practice, each convolutional layer is followed by batch normalization, and a ReLU is applied after the addition.

In this work, we define a residual block as follows.

Definition 2.9 (Residual Block). *A residual block $b = \{l_i, \dots, l_j\} \subset \mathcal{L}$ is a group of consecutive layers that consists of two paths. The main path transforms the input $\mathbf{x}$ through the sequence of layers and computes the residual function $F(\mathbf{x})$. The skip connection passes the input $\mathbf{x}$ unchanged. The output of the block is the sum of both paths: $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$. We denote the set of all residual blocks in a network as $\mathcal{B}$.*

There are two types of skip connections. The *identity shortcut* passes the input x through unchanged, as shown in Figure 2.5. This is used when the input and output dimensions are the same. When the dimensions differ, a *projection shortcut* is used instead. It applies a transformation to x to match the output dimensions before the addition.

Besides the different types of skip connections, there are also different types of residual blocks [8]. A *BasicBlock* consists of two convolutional layers, each followed by batch normalization, with a ReLU activation between them. It is used in shallower residual networks. A *Bottleneck* block is larger and consists of three convolutional layers, each followed by batch normalization. The name comes from its structure: the first layer reduces the channel dimension, the computation is performed on this smaller representation, and the third layer restores the original dimension. It is used in deeper residual networks.

By stacking multiple residual blocks, a residual network is formed. We define it as follows.

Definition 2.10 (Residual Network). *A residual network is a convolutional neural network $\mathcal{M} = (\mathcal{N}, \mathcal{S})$ whose layers $\mathcal{L}$ are organized into a sequence of residual blocks $\mathcal{B} = \{b_1, \dots, b_K\}$. The blocks in $\mathcal{B}$ are either all BasicBlocks or all BottleneckBlocks.*

He et al. [8] presented several ResNet variants, named by their total number of layers. Table 2.1 shows the complete list of variants. Shallower models like ResNet-18 and ResNet-34 use BasicBlocks, while deeper models starting from ResNet-50 use BottleneckBlocks. The blocks are organized into four groups, each operating at a different spatial resolution.

Model	Block Type	Blocks per Group	Total Layers
ResNet-18	BasicBlock	2, 2, 2, 2	18
ResNet-34	BasicBlock	3, 4, 6, 3	34
ResNet-50	Bottleneck	3, 4, 6, 3	50
ResNet-101	Bottleneck	3, 4, 23, 3	101
ResNet-152	Bottleneck	3, 8, 36, 3	152

Table 2.1: ResNet variants with their block types and configurations [8]. Shallower models use BasicBlocks, deeper models use BottleneckBlocks

With these definitions, we have introduced all network structures and notation relevant for the following chapters. In this work, we exclusively focus on residual networks. Besides understanding the network architecture, our approach also builds on the concept of program slicing, which we introduce in the next section.

2.4 Program Slicing

Program slicing is a technique for extracting relevant parts of a program with respect to a specific computation [19, 3]. Given a variable of interest at a specific point in the program, a slice contains only those statements that may affect the value of that variable. We formalize this as follows.

Definition 2.11 (Program). *A program $P = \{s_1, s_2, \dots, s_n\}$ is a set of statements, where each statement s_i represents an individual instruction.*

Since a program consists of multiple statements, we can define a slice as a subset of these statements that are relevant to a specific computation.

Definition 2.12 (Program Slice). *Let P be a program and $C = (s, V)$ a slicing criterion, where s is a statement in P and V is a set of variables. A program slice $P_C \subseteq P$ is the set of all statements in P that may affect the values of variables in V at statement s .*

Two main variants of program slicing are distinguished in the literature:

- **Static slicing:** Computes slices by analyzing the source code without executing it. The resulting slice is valid for all possible inputs to the program [3].
- **Dynamic slicing:** Computes a slice dynamically at runtime during program execution. The resulting slice is based on a specific input and only contains statements that were actually executed [20].

We illustrate both program slicing variants using the example program P in Listing 2.1. The program consists of eight statements in total. It reads an input value, performs a conditional computation and finally prints the result. The slicing criterion is $C = (8, \{z\})$, where $s = 8$ refers to the last statement `print(z)` as the point of interest. The variable we want to observe is $V = \{z\}$. Both variants then determine which statements in the program affect the value of z at that point.

Listing 2.1: Example program P with slicing criterion $C = (8, \{z\})$.

```
x = input()
y = 5
w = 10
if x > 0:
    z = x + y
else:
    z = x - y
print(z)
```

For the static slice, the source code is analyzed without executing it. Since x is read from user input, its value is unknown and could be any possible value. Therefore, both the `if`-branch where $x > 0$ and the `else`-branch where $x \leq 0$ must be considered. However, line 3 (`w = 10`) does not affect z and is excluded. The static slice therefore includes all statements in P except for line 3: $P_C = \{1, 2, 4, 5, 6, 7, 8\}$.

Listing 2.2: Static slice $P_C = \{1, 2, 4, 5, 6, 7, 8\}$ for $C = (8, \{z\})$.

```

x = input()
y = 5
w = 10
if x > 0:
    z = x + y
else:
    z = x - y
print(z)

```

A dynamic slice considers a specific execution with a known input such as $x = 3$. Since the input is known, only the statements that were actually executed and affected z are included. With $x = 3 > 0$, only the `if`-branch is taken, and both line 3 and the `else`-branch are excluded. The dynamic slice is therefore the following set of statements: $P_C = \{1, 2, 4, 5, 8\}$.

Listing 2.3: Dynamic slice $P_C = \{1, 2, 4, 5, 8\}$ for $C = (8, \{z\})$ with input $x = 3$.

```

x = input()
y = 5
w = 10
if x > 0:
    z = x + y
else:
    z = x - y
print(z)

```

Compared to the static slice, the dynamic slice is smaller because it only considers the execution path that was actually taken for the given input $x = 3$. In general, dynamic slices are subsets of static slices and provide a more precise view of the program behavior for a specific execution.

Over the years, various application areas for program slicing on traditional programs have been explored. For example, dynamic slicing can be used in debugging to locate faulty statements by narrowing down the code that could have caused an incorrect value [21]. Similarly, slicing has been applied to testing, where it can reduce test suites by identifying and removing test cases that exercise the same program paths [22].

The concept of dynamic slicing has also been extended beyond traditional software. Zhang et al. [6] applied it to neural networks. Instead of a program P with a set of statements, we now consider a neural network $\mathcal{M} = (\mathcal{N}, \mathcal{S})$ with a set of neurons and synapses. The slicing criterion also changes: instead of a statement and a set of variables, it consists of a set of input samples I and a set of output neurons O . The resulting slice is the subset of neurons and synapses that significantly contribute to the output. A neural network slice is defined in Definition 2.13. We adopt this definition for the rest of this work.

Definition 2.13 (Neural Network Slice [6]). *Let $\mathcal{M} = (\mathcal{N}, \mathcal{S})$ be a neural network and $C = (I, O)$ a slicing criterion, where $I = \{\xi_1, \xi_2, \dots, \xi_m\}$ is a set of input samples and $O = \{o_1, o_2, \dots, o_k\}$ is a set of output neurons of interest. The neural network slice $\mathcal{M}_C = (\mathcal{N}_C, \mathcal{S}_C)$ with $\mathcal{N}_C \subseteq \mathcal{N}$ and $\mathcal{S}_C \subseteq \mathcal{S}$ is the subset of neurons and synapses that significantly contribute to the value of any output $o \in O$ for any input $\xi \in I$.*

For example, given an input image ξ of a truck and the output neuron o for the class *truck*, the slice $\mathcal{M}_C$ identifies which neurons and synapses in the network were responsible for classifying ξ as a truck.

In this chapter, we introduced the foundations of neural networks and presented program slicing as a technique for identifying relevant parts in traditional programs. We showed how the concept extends to neural networks, where slices identify neurons and synapses relevant to a specific prediction. The definitions and notation established here form the basis for the following chapters. The next chapter reviews related work in program slicing, neural network slicing, and model interpretability.

3 Related Work

This chapter reviews existing work related to our thesis. We begin with approaches that apply slicing techniques to neural networks. After that, we discuss interpretability methods that aim to explain model predictions. We conclude with program analysis techniques that have been adapted to test and analyze neural networks.

3.1 Neural Network Slicing

Program slicing has recently been adapted to deep neural networks [6, 5, 23, 7]. The goal shifts from extracting traditional program statements that affect a variable of interest to identifying the internal components of a network that contribute to a particular output.

NNSlicer applies dynamic program slicing to deep neural networks [6]. In a profiling phase, it estimates the average activation of each neuron over the training data. A forward analysis then computes activation deltas for a specific input. Finally, a backward analysis propagates these deltas from the output layer back through the network to obtain a slice. The authors demonstrate that slices can support tasks such as detecting adversarial inputs, pruning models, and protecting specific predictions. However, NNSlicer operates exclusively at the individual neuron level. Modern convolutional neural networks are organized into higher-level structures such as channels and residual blocks. Our work extends NNSlicer by aggregating neuron-level information to channels and residual blocks. We perform slicing at these coarser architectural granularities, which have not been explored in prior work.

Other work extracts critical paths through a network instead of explicit slices. Qiu et al. propose effective paths, which treat the network as a data-flow graph [23]. For each input, they select the minimal set of neurons and synapses whose positive contributions exceed a fixed fraction of the output logit. Only positive contributions are considered, meaning that neurons which suppress the output are excluded. By aggregating per-input paths into class-level paths, they show that different classes activate distinct sub-networks and use path similarity for adversarial example detection. In contrast, our approach builds on NNSlicer, which considers both positive and negative deviations from the mean activation. Similar to NNSlicer, effective paths operate exclusively at the neuron level, whereas our method introduces a hierarchical block and channel structure above the neuron level.

Wang et al. interpret neural networks by identifying critical data routing paths through learned control gates [5]. Each gate is a non-negative scalar attached to a channel, where a high value indicates a critical channel and a low value indicates a less important one. This is conceptually similar to our channel-level deltas, which also assign a single value per channel to determine its relevance. However, the mechanisms differ. Their gates must be trained separately for each individual input, which adds substantial computational cost. Our deltas, in contrast, are derived directly from

existing activations without additional training.

A different line of work targets model maintenance rather than interpretability. NeuSemSlice introduces semantic slicing, which identifies critical neurons based on contribution scores [7]. For each category and layer, it selects a subset of neurons whose representations retain high centered-kernel-alignment (CKA) similarity to the full layer. The selected neurons are then merged across categories into semantic components that support maintenance tasks such as pruning, fine-tuning, and incremental development. Unlike our approach, NeuSemSlice measures neuron relevance through CKA similarity across the training set, whereas we compute per-input activation deltas relative to a profiled mean. Overall, NeuSemSlice is designed for semantic-aware model maintenance, whereas our work focuses on efficient slicing of residual networks.

3.2 Model Interpretability

Explainable artificial intelligence (XAI) aims to explain predictions of deep neural networks and make their decisions more transparent to humans [2, 24]. Nazir et al. provide a recent taxonomy of feature attribution techniques and distinguish between model-agnostic and model-specific approaches [25]. Model-agnostic methods treat the network as a black box and operate only on its inputs and outputs. Model-specific methods, on the other hand, leverage the internal structure and parameters of the network and can therefore provide explanations that are based on internal computations. Our work falls into the second category, as we trace activation patterns through the internal layers of convolutional networks to extract class-specific slices.

Among model-specific methods, reference-based approaches explain predictions by comparing neuron activations against a reference input. DeepLIFT computes contribution scores for each neuron by measuring the difference between its activation and a reference activation [26]. These scores are then propagated backward through the network to quantify how much each neuron contributed to the output. This neuron-level perspective is conceptually close to our approach, as we also compute per-neuron activation differences. However, DeepLIFT requires a fixed reference input and explains individual predictions, whereas we aggregate activations over many samples of a class to obtain a class-level view.

Rather than comparing against a reference, activation-based methods directly use the feature maps produced during inference. Grad-CAM computes class-specific localization maps by weighting the feature maps of a convolutional layer with the gradients flowing back from the target class [27]. The resulting heatmap highlights which spatial regions of the input are most relevant for a given prediction. Our approach is also activation-based, as we collect neuron activations via forward hooks during inference. Instead of producing spatial heatmaps, we aggregate these activations to identify which neurons, channels, and residual blocks are critical for a given class.

A third line of work propagates relevance scores rather than activations or gradients. Layer-wise relevance propagation (LRP) redistributes the prediction score backward

through the network by applying local propagation rules at each layer [28]. This is conceptually similar to our backward analysis, where we also apply operation-specific rules layer by layer to propagate the contribution of each neuron to the final slice. However, LRP decomposes a single prediction down to the input features, while our backward pass traces contributions across neurons, channels, and blocks to extract class-specific slices.

3.3 Program Analysis for Neural Networks

Beyond slicing and interpretability, ideas from traditional program analysis have been applied to neural networks for testing, fault localization, and model repair. Unlike traditional program analysis, which operates on source code, the following methods work directly on the trained network by analyzing its internal activations and parameters.

One line of work adapts fault localization techniques to identify neurons that cause incorrect predictions. DeepLocalize monitors internal activations during training and flags neurons whose values deviate from expected ranges [29]. Hashemifar et al. transfer spectrum-based fault localization to neural networks and analyze execution paths to rank suspicious neurons [30]. Both approaches operate at the neuron level and trace information through network layers, which is structurally similar to our forward and backward analysis. However, their goal is to locate faulty neurons that cause mispredictions, whereas we identify neurons that are relevant for a specific class.

Complementary to fault localization, neural network testing applies coverage criteria to measure how thoroughly a test suite exercises a model. DeepXplore introduces neuron coverage and uses differential testing across multiple models to generate inputs that trigger previously inactive neurons [31]. DeepGauge extends this with finer-grained metrics at the neuron and layer level, such as k-multisection coverage and boundary coverage [32]. These methods share the idea of reasoning about individual neuron activations, but they measure test adequacy rather than class-specific relevance.

A third line of work applies neuron-level modifications to repair undesirable model behavior once faults or unfairness have been detected. RUNNER identifies neurons responsible for unfair predictions and adjusts their weights to reduce bias [33]. Fayyazi et al. use counterexample-guided verification to locate and repair fairness violations in neural networks [34]. While these methods also operate on individual neurons, they modify the network to correct its behavior. Our approach does not alter the model but extracts a subset of it as a standalone slice.

The work in this thesis builds on the data-flow-based approach of NNSlicer and extends it with coarser slicing granularities. While interpretability methods like DeepLIFT and Grad-CAM explain which input features are important, our approach answers a complementary question: which internal components of the network are responsible for a prediction. Similar to Grad-CAM, we operate at the feature-map level, but instead of highlighting spatial regions, we identify which channels and blocks

contribute to a decision. In contrast to testing, fault localization, and repair, we use slicing purely as a descriptive tool to extract reusable, class-specific subnetworks. To the best of our knowledge, this is the first approach that extends neuron-level slicing to a hierarchical block-based method.

4 NNSlicer

The following chapter presents a detailed overview of the NNSlicer framework that was introduced by Zhang et al. [6]. NNSlicer adapts the concepts of dynamic program slicing to neural networks by calculating the neurons and synapses that contribute most to a specific output. Understanding this framework is essential, as our block-based slicing approach builds directly on its pipeline and contribution rules.

NNSlicer consists of three phases to construct a neural network slice (Definition 2.13). In the first *profiling phase*, the mean activation value of each neuron is computed over the entire training dataset $\mathcal{I}$. These means establish a baseline of typical neuron behavior across the dataset. The second *forward analysis phase* calculates the activation values for a specific input sample $\xi \in \mathcal{I}$ and compares them to the mean baseline from profiling. The resulting activation deltas represent how much each neuron reacts to the input. In the final *backward analysis phase*, the actual slices are constructed. Contributions are propagated backward from the target output neurons to the input layer. The result is a slice $\mathcal{M}_C = (\mathcal{N}_C, \mathcal{S}_C)$, i.e., the subset of neurons and synapses that significantly contribute to the output. In the following sections, we take a closer look at each phase with visualized examples.

4.1 Profiling

The profiling phase establishes a baseline for neuron behavior. Recall that each neuron $n \in \mathcal{N}$ produces an activation value y_n for a given input. We write $y_n(\xi)$ for the activation of neuron n on a specific input sample ξ , and define its mean activation over the training dataset $\mathcal{I}$ as:

$$\bar{y}_n = \frac{1}{|\mathcal{I}|} \sum_{\xi \in \mathcal{I}} y_n(\xi) \quad (4.1)$$

This mean $\bar{y}_n$ captures the typical response of each neuron across all inputs and serves as a reference point for the forward analysis, where input-specific activations are compared against it. Since the mean is computed over the entire dataset $\mathcal{I}$, profiling requires one forward pass per sample. However, profiling only needs to be computed once per model and dataset and can be reused for all subsequent slicing operations.

Figure 4.1 illustrates profiling for a simple network. On the left, the pretrained model contains only the learned weights on its connections. After profiling, each neuron stores its mean activation value computed over the training data. For example, the first hidden neuron has a mean activation of $\bar{y}_n = 2$, meaning it produces a mean activation of 2 across all training samples in $\mathcal{I}$.

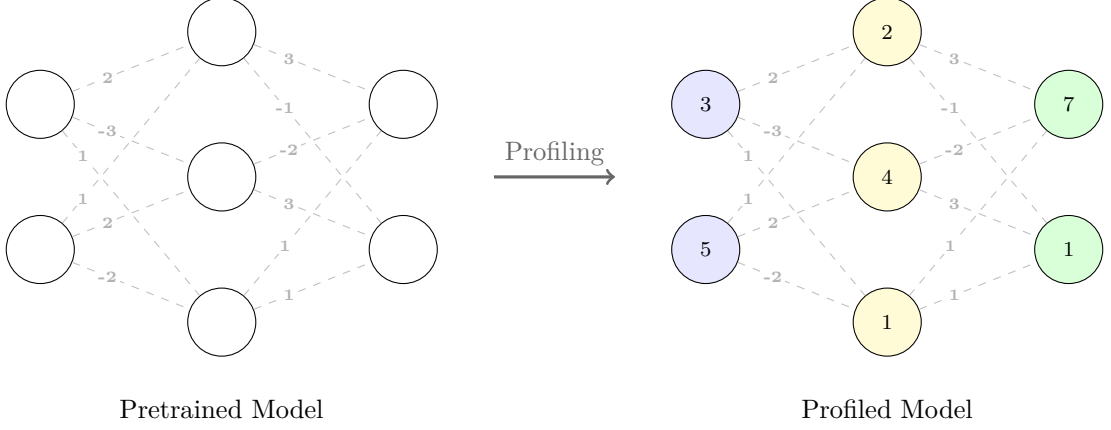


Figure 4.1: Profiling phase. Mean activations $\bar{y}_n$ are computed for every neuron $n \in \mathcal{N}$ over the training dataset $\mathcal{I}$

4.2 Forward Analysis

Given the profiled model, the forward analysis phase records activation values for a specific input sample $\xi \in \mathcal{I}$ and compares them to the baseline from profiling. For each neuron $n \in \mathcal{N}$, the activation delta is computed as

$$\Delta_n = y_n(\xi) - \bar{y}_n, \quad (4.2)$$

where $y_n(\xi)$ is the activation of neuron n for the current input sample ξ and $\bar{y}_n$ is its mean activation from profiling. These deltas capture how much the response of a neuron deviates from its typical behavior. They are used in the backward analysis to weight the contribution of each neuron, so that neurons with stronger reactions to the input have a larger influence on the resulting slice. Figure 4.2 illustrates this process for an input $\xi = [1, 2]$.

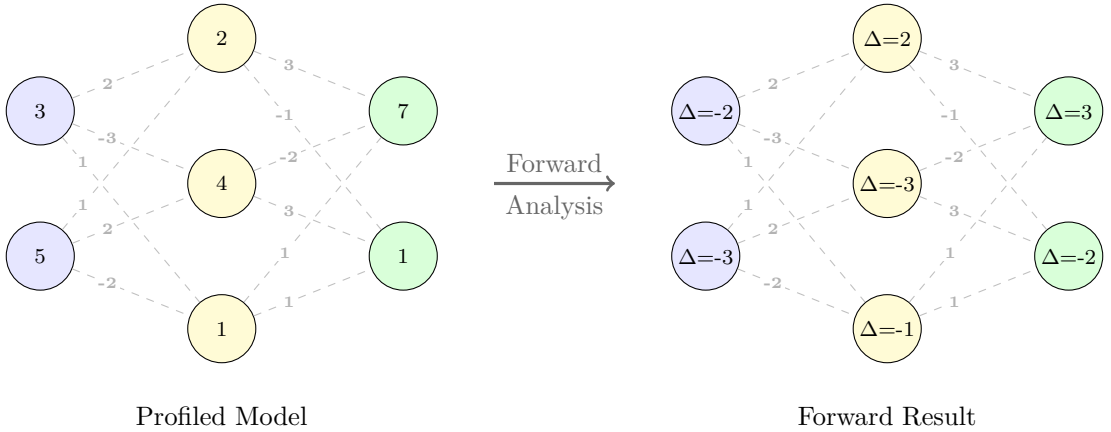


Figure 4.2: Forward analysis phase. Activation deltas $\Delta_n = y_n(\xi) - \bar{y}_n$ are computed for a specific input $\xi = [1, 2]$

The profiled model on the left shows the mean activations from Figure 4.1. After processing the input, the activation deltas are computed by subtracting the mean values from the observed activations. For example, the first hidden neuron has an observed activation of 4 and a mean activation of 2, yielding a delta of $\Delta = 4 - 2 = 2$. A positive delta means the neuron is more active than the mean across the dataset, while a negative delta means it is less active. In both cases, a larger absolute value indicates a stronger deviation from the baseline. The absolute value of the delta $|\Delta_n|$ is called its magnitude and can be interpreted as the sensitivity of neuron n to the input.

4.3 Backward Analysis

The backward analysis is the final phase, in which the contribution of each neuron and synapse to the target output neurons O is computed. Note that the delta of a neuron is not the same as its contribution to a specific output. A neuron may react strongly to an input without contributing to the target prediction. For example, a neuron that detects wing shapes may have a high delta for a given image of a bird. However, when computing the slice for the output class automobile on this image, the activation of the neuron does not influence the automobile prediction and has no contribution.

To compute these contributions, NNSlicer considers each neuron n and its k predecessor neurons $n_1, \dots, n_k$ that are connected to n via synapses. For each predecessor, a contribution value is computed that describes how much it influences n . NNSlicer distinguishes between two types of contributions:

- The *local contribution* $contrib_i$ is generated by a single operation and describes how much a preceding neuron n_i influences the current neuron n in that computation.
- The *cumulative contribution* $CONTRIB_n$ accumulates all local contributions that neuron n receives during the backward pass, representing its overall importance for the target output.

Initially, the cumulative contribution of the target output neuron $o \in O$ is set to $CONTRIB_o = 1$. The backward phase then recursively processes the network from the output layer back to the input layer. At each layer l , local contributions $contrib_i$ are calculated for all predecessor neurons $n_i \in \mathcal{N}_{l-1}$. These are accumulated into $CONTRIB$, and neurons with $CONTRIB_{n_i} \neq 0$ become the new target set. This process repeats until no more predecessors remain. Algorithm 1 describes the general workflow.

The computation of the local contribution $contrib_i$ depends on the type of layer. Different layers perform different operations, and each requires its own contribution rule. For the weighted sum operation, which is used in both convolutional and fully connected layers, the local contribution of the i -th predecessor to a neuron n is:

$$contrib_i = CONTRIB_n \cdot \Delta y \cdot w_i \Delta x_i \quad (4.3)$$

Algorithm 1 ComputeContrib [6]: computing the contributions of neurons and synapses for an input sample

Require: Neural network $\mathcal{M} = (\mathcal{N}, \mathcal{S})$, input sample ξ , target neurons O

```

1: if  $O$  is empty then
2:   terminate
3: end if
4: Initialize  $O' \leftarrow \emptyset$ 
5: for each neuron  $o \in O$  do
6:   Find  $o$ 's preceding neurons  $(\mathcal{N}', \mathcal{S}')$ 
7:   Compute local contributions of  $\mathcal{N}'$  and  $\mathcal{S}'$  as contrib
8:   Update CONTRIB with contrib
9: end for
10: for each neuron  $n \in \mathcal{N}$  do
11:   Add  $n$  to  $O'$  if  $n$  is a predecessor of  $O$  and  $CONTRIB_n \neq 0$ 
12: end for
13: Obtain  $\mathcal{N}'$  by removing neurons in  $O$  from  $\mathcal{N}$ 
14: call COMPUTECONTRIB with  $O \leftarrow O'$  and  $\mathcal{N} \leftarrow \mathcal{N}'$ 
15: return CONTRIB

```

where Δy is the activation delta of neuron n , Δx_i the activation delta of the preceding neuron n_i , and w_i the synapse weight. The product $\Delta y \cdot w_i \Delta x_i$ captures the impact that predecessor n_i has on n . For example, if Δy is negative and $w_i \Delta x_i$ is positive, then n_i reinforces the negative deviation of n , yielding a contribution that is opposite in sign to $CONTRIB_n$.

Next to the operation rule of weighted sum, NNSlicer defines four more contribution rules for common neural network operations. All supported layers and their corresponding contribution rules are listed in Table 4.1.

Table 4.1: Backward contribution rules for each operator type. $CONTRIB_n$ is the cumulative contribution of neuron n . Δy , Δx_i are the activation deltas of the output and i -th input neuron, respectively [6]

Operation	Usage	Definition	Contribution of i -th input
Weighted sum	Conv./FC layers	$y = \sum_{i=1}^k w_i x_i$	$CONTRIB_n \cdot \Delta y \cdot w_i \Delta x_i$
Average	Avg-pooling	$y = \frac{1}{k} \sum_{i=1}^k x_i$	$CONTRIB_n \cdot \Delta y \cdot \Delta x_i$
Maximum	Max-pooling	$y = \max_{i=1}^k x_i$	$CONTRIB_n \cdot \Delta y \cdot \Delta x_i$ if $x_i = y$, else 0
Rectify	ReLU	$y = \max(x, 0)$	$CONTRIB_n \cdot \Delta y \cdot \Delta x$ if $x > 0$, else 0
Scale	Batch norm.	$y = \frac{x - \mu}{\sigma}$	$CONTRIB_n \cdot \Delta y \cdot \Delta x$

ReLU, batch normalization, and pooling layers have a simpler structure than con-

voluntional and fully connected layers. They each take a single input and apply a fixed transformation. Their contribution rules reflect this by directly passing the contribution through, scaled by the activation deltas.

Since different layer types produce contributions at different scales, local contributions can vary greatly in magnitude. A fully connected layer with large weights may produce much larger contributions than a normalization layer. To prevent any single value from dominating the cumulative result, NNSlicer accumulates only the sign of each local contribution:

$$CONTRIB_{n_i} += \text{sign}(\text{contrib}_i), \quad CONTRIB_{s_i} += \text{sign}(\text{contrib}_i) \quad (4.4)$$

In this way, *CONTRIB* reflects how consistently a neuron contributes across operations, rather than how strongly it contributes in any single one. A neuron is considered important if $CONTRIB_n \neq 0$, and it may contribute positively or negatively to the slicing criterion.

Accumulating these contributions across a network with many neurons can be time-consuming. NNSlicer reduces this cost by excluding neurons with small contributions. It introduces a threshold parameter θ that controls how many local contributions are included. Contributions are sorted by magnitude, and the smallest ones are excluded as long as their cumulative influence on the activation of the neuron remains below θ .

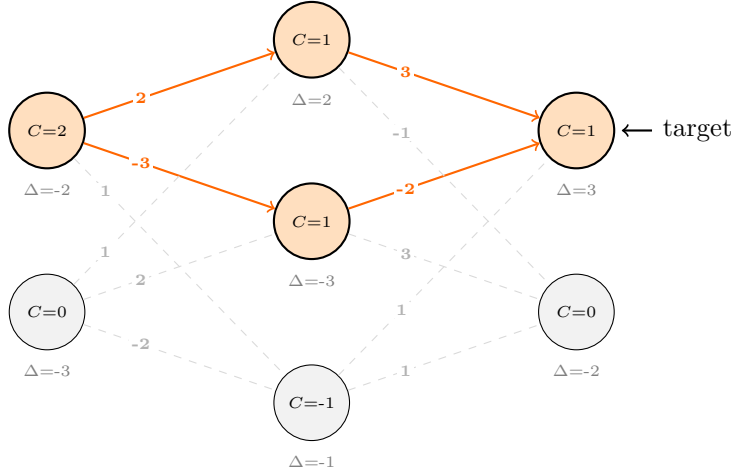


Figure 4.3: Backward analysis phase. Contributions are traced backward from the target neuron. Orange nodes and edges form the computed slice, gray elements are excluded

To illustrate these concepts, we apply the backward analysis to the same example network from the previous phases. We use a threshold of $\theta = 0.2$ throughout the example. The resulting slice for the target output neuron o_1 is shown in Figure 4.3, where orange neurons and synapses are included in the slice and gray ones are excluded.

Following Algorithm 1, the backward analysis starts by setting $CONTRIB_{o_1} = 1$. The target output neuron o_1 has $\Delta y = 3$ and is connected to three predecessor neurons

in the hidden layer via synapses with weights $w_1 = 3$, $w_2 = -2$, and $w_3 = 1$. Since the network consists of fully connected layers, the weighted sum contribution rule is applied. Before computing the local contributions, the threshold θ is applied. The connection between o_1 and the first hidden neuron has a magnitude of $|w_1 \cdot \Delta x_1| = |3 \cdot 2| = 6$. For the second hidden neuron, the magnitude is $|w_2 \cdot \Delta x_2| = |(-2) \cdot (-3)| = 6$. The third has a much smaller magnitude of $|w_3 \cdot \Delta x_3| = |1 \cdot (-1)| = 1$. The total magnitude across all three connections is $6 + 6 + 1 = 13$. With $\theta = 0.2$, the smallest contributions can be excluded as long as their share of the total remains below θ . Since the third connection accounts for only $1/13 < 0.2$, it is excluded. For the two remaining predecessor neurons, the local contributions are computed with the weighted sum contribution rule. For the first hidden neuron, the local contribution is $contrib_1 = 1 \cdot 3 \cdot 3 \cdot 2 = 18$. For the second hidden neuron, the local contribution is $contrib_2 = 1 \cdot 3 \cdot (-2) \cdot (-3) = 18$. The cumulative contributions are then updated by adding the sign of the local contributions: $CONTRIB_{h_1} += \text{sign}(18) = +1$ and $CONTRIB_{h_2} += \text{sign}(18) = +1$. Both hidden neurons are added to the slice, while the third hidden neuron with $CONTRIB = 0$ is excluded. The target set is then updated to $O' = \{h_1, h_2\}$ and the process repeats for the next layer. This continues until all layers have been processed. The neurons with $CONTRIB \neq 0$ and the synapses connecting them form the resulting slice $\mathcal{M}_C = (\mathcal{N}_C, \mathcal{S}_C)$.

This chapter presented the three phases of NNSlicer and illustrated them with a concrete example to create a slice for a target output neuron. While NNSlicer operates at the level of individual neurons and synapses, the next chapter presents our block-based slicing approach and concludes with the vectorized implementation.

5 Block-Based Dynamic Slicing

In this chapter, we present our block-based slicing approach, which extends the neuron-level analysis of NNSlicer to operate at a coarser granularity. *Block slicing* determines which residual blocks to retain from all blocks in the network. *Channel slicing* then selects which channels to retain within the remaining blocks. Only after both filtering steps, the neuron-level backward analysis is performed on the remaining components. We extend the NNSlicer workflow with the following steps. They are executed in the listed order, forming a hierarchical pipeline that works from the coarsest level down to individual neurons:

1. **Forward analysis with delta aggregation:** During the forward analysis, activation deltas are recorded for each neuron. We aggregate these deltas into channel-level, layer-level, and block-level deltas.
2. **Block slicing:** The block deltas are used to identify residual blocks that show little response to the current input. These blocks are skipped during the backward analysis.
3. **Channel slicing:** Within the retained blocks, channel deltas are used to select the most responsive channels in each convolutional layer.
4. **Vectorized backward analysis:** For the remaining neurons in the retained channels and blocks, the backward analysis is performed using vectorized tensor operations.

The chapter is structured as follows. The first section describes the delta aggregation procedure, which provides the basis for the filtering steps. The next two sections present block slicing and channel slicing, respectively. The chapter concludes with the vectorized backward analysis.

5.1 Delta Aggregation

The backward analysis is the most computationally expensive phase of NNSlicer, as it recursively propagates contributions through neurons layer by layer. For every individual input sample ξ and output neuron o , the backward analysis has to be repeated across the entire network to create the slice for that target. As a network grows in size and complexity, the number of neurons increases and the backward analysis becomes increasingly expensive. Our goal is to identify irrelevant components before the backward analysis begins, so that the computation for those parts of the network can be skipped entirely.

NNSlicer performs two phases before backward analysis: profiling and forward analysis. Profiling computes the mean activation $\bar{y}_n$ for each neuron over the entire dataset $\mathcal{I}$. The computational cost of profiling is also high, as it requires a forward

pass for each training sample $\xi \in \mathcal{I}$ to capture the activations. This cost increases with the size of the dataset to be profiled. In contrast to the backward analysis, profiling only needs to be computed once and can be reused for all inputs. However, profiling is not suitable as a filtering criterion in our case. The resulting averages are dataset-wide and do not provide information specific to an individual input. Since we perform dynamic slicing, we need input-specific values to determine which components are relevant for a given prediction.

The forward analysis provides exactly this. It computes activation deltas Δ_n for every neuron, which measure how much the response of the neuron for the current input deviates from its dataset average. A large deviation from the average, whether positive or negative, suggests that the neuron reacts strongly to this particular input. A delta close to zero means it behaves similar to its average and is unlikely to carry input-specific information. We extend this idea to coarser structures. Since neural networks are organized into channels, layers, and blocks, we can aggregate neuron-level deltas into a single value per structure. This gives us a criterion that is available right after the forward pass and before we start the backward analysis.

The hierarchical delta aggregation is shown in Figure 5.1. Neuron deltas are aggregated into channel deltas. Channel deltas are aggregated into layer deltas. Finally, layer deltas are aggregated into block deltas.

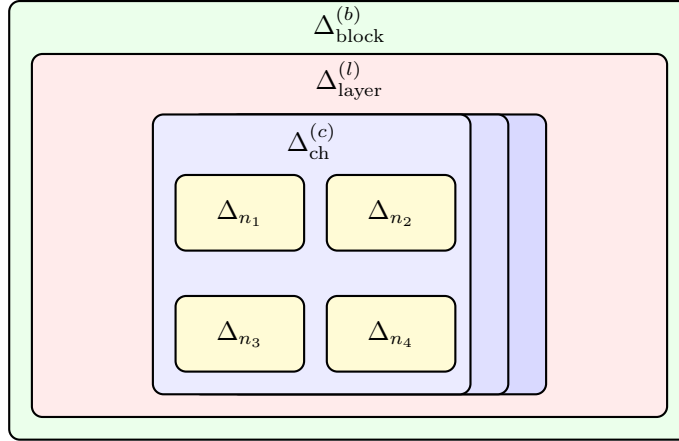


Figure 5.1: Hierarchical delta aggregation. Neuron deltas are aggregated into channel deltas, then into layer deltas, and finally into block deltas

We now define each delta level from bottom to top, starting with channels and ending with blocks. Recall that a channel c consists of a spatial grid of neurons $\mathcal{N}_{\text{ch}}$ that share the same kernel weights. By aggregating the neuron-level deltas within a channel, we obtain a single value that describes how much the activation of the channel deviates from its mean behavior. We use the absolute value of the delta because we are interested in how much the behavior of a neuron deviates from its average, regardless of direction. A positive delta means the neuron is more active than usual for the given input, while a negative delta means it is less active. Both directions indicate that the neuron reacts to the input. Without the absolute value,

positive and negative deviations within the same channel could cancel each other out, making an active channel appear inactive.

We take the mean rather than the sum to make the channel delta independent of the spatial dimensions. This ensures that channels of different sizes can be compared fairly. For example, in deeper layers of a convolutional network, the spatial dimensions shrink due to pooling. Using the sum, channels in earlier layers with larger spatial dimensions would systematically receive higher values than channels in deeper layers, regardless of how strongly their neurons react. We therefore define the channel delta as the mean of the absolute neuron deltas within the channel.

Definition 5.1 (Channel Delta). *For a channel c with neurons $\mathcal{N}_{ch} = \{n_{i,j} \mid 1 \leq i \leq H, 1 \leq j \leq W\}$ arranged in a spatial grid of height H and width W , the channel delta is the mean absolute neuron delta over all neurons in the channel:*

$$\Delta_{ch} = \frac{1}{|\mathcal{N}_{ch}|} \sum_{n \in \mathcal{N}_{ch}} |\Delta_n| \quad (5.1)$$

Consider a channel with four neurons whose deltas are $\Delta_n = 0.9, -0.7, 0.8, -0.6$. The channel delta is $\Delta_{ch} = \frac{1}{4}(|0.9| + |-0.7| + |0.8| + |-0.6|) = 0.75$. Note that both positive and negative deviations contribute equally, since the absolute value ensures they do not cancel out.

The same idea of aggregation can be applied at the layer level. A convolutional layer l consists of a set of channels $\mathcal{C}$. The layer delta is the mean over all channel deltas $\Delta_{ch}^{(c)}$ of its channels $c \in \mathcal{C}$. As with the channel delta, we use the mean to ensure that layers with different numbers of channels are comparable. For fully connected layers without channels, the layer delta is instead computed directly as the mean over the absolute neuron deltas $|\Delta_n|$ of its neurons $n \in \mathcal{N}_l$.

Definition 5.2 (Layer Delta). *For a convolutional layer l with channels $\mathcal{C} = \{c_1, \dots, c_n\}$, the layer delta is the mean of the channel deltas of its channels:*

$$\Delta_{layer} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \Delta_{ch}^{(c)} \quad (5.2)$$

For a fully connected layer l with neurons $\mathcal{N}_l = \{n_1, \dots, n_m\}$, the layer delta is defined as:

$$\Delta_{layer} = \frac{1}{|\mathcal{N}_l|} \sum_{n \in \mathcal{N}_l} |\Delta_n| \quad (5.3)$$

A high layer delta means that many of its channels deviate from their average behavior. A possible interpretation is that an early layer detecting edges and textures may have a high delta for images with strong visual patterns but a low delta for uniform backgrounds.

Note that we do not use layer deltas directly as a filtering criterion. Removing an entire layer from the network would break the data flow, as subsequent layers depend on its output. Residual blocks, by contrast, include a skip connection that passes the

input unchanged. This allows skipping a block without interrupting the flow through the network. Layer deltas serve instead as an intermediate step to aggregate channel deltas into block deltas.

A residual network organizes its layers into a set of residual blocks $\mathcal{B} = \{b_1, \dots, b_K\}$, where each block $b \in \mathcal{B}$ groups multiple consecutive layers. We average their layer deltas $\Delta_{\text{layer}}^{(l)}$ for each layer $l \in b$ into a single block delta.

Definition 5.3 (Block Delta). *For a residual block $b = \{l_1, \dots, l_n\}$, the block delta is the mean of the layer deltas of its layers:*

$$\Delta_{\text{block}} = \frac{1}{|b|} \sum_{l \in b} \Delta_{\text{layer}}^{(l)} \quad (5.4)$$

Each residual block acts as a functional unit within the network. A large block delta indicates that this unit as a whole is active for the current input. For instance, a block that is sensitive to object shapes could show a high delta for object images but remain close to zero for scenes dominated by textures.

With these aggregated deltas, we now have an indicator for both channels and blocks that suggests which components may be relevant for a given input. The remaining question is how to decide which components to retain before the backward analysis starts. The following two sections describe this selection process in detail: first at the block level, then at the channel level.

5.2 Block Slicing

The idea behind block slicing is to slice the network at its coarsest granularity before the backward analysis begins. As described in the previous section, blocks with high deltas indicate strong reactions to the current input, while blocks with deltas close to zero are candidates for skipping. To select which blocks to retain, we adopt a cumulative energy filtering strategy. We define the total energy as the sum of all block deltas.

Definition 5.4 (Block Energy). *The block energy is the sum of block deltas over all blocks $\mathcal{B}$:*

$$E_{\text{block}} = \sum_{b \in \mathcal{B}} \Delta_{\text{block}}^{(b)} \quad (5.5)$$

This gives us a measure of the overall response of the network to the current input at the block level. A block with a high delta contributes a large share to the total energy, while a block with a low delta contributes less.

We introduce a new hyperparameter $\beta \in [0, 1]$ to control this filtering process. First, we sort all blocks by their delta in descending order. We then accumulate deltas starting from the block with the highest delta until the cumulative sum reaches a fraction β of E_{block} . All remaining blocks are candidates for skipping. A lower value of β means that less cumulative energy is required, so fewer blocks are retained and

more are skipped. This implies a monotonicity property: lowering β always produces a subset of the blocks retained at a higher β , since the algorithm accumulates blocks in the same sorted order. Setting β to 1.0 requires the full energy to be accumulated, which means all blocks are retained. This is equivalent to the original NNSlicer without block filtering. The strategy itself is inspired by the threshold θ in the backward analysis of NNSlicer, where contributions are similarly sorted by magnitude and retained until a fraction of the total is reached. We chose this approach to maintain a consistent filtering strategy across all levels of the hierarchy. Algorithm 2 formalizes the block slicing process based on cumulative delta energy.

Algorithm 2 BlockSlice: identifying residual blocks to skip

Require: Block deltas $\Delta_{\text{block}}^{(b)}$ for each $b \in \mathcal{B}$, total energy E_{block} , threshold $\beta \in [0, 1]$
Ensure: Set of blocks to skip *skip*

- 1: Sort blocks by $\Delta_{\text{block}}^{(b)}$ in descending order
- 2: $\text{accumulated} \leftarrow 0$
- 3: $\text{keep} \leftarrow \emptyset$
- 4: **for** each block b in sorted order **do**
- 5: $\text{accumulated} \leftarrow \text{accumulated} + \Delta_{\text{block}}^{(b)}$
- 6: Add b to *keep*
- 7: **if** $\text{accumulated} \geq \beta \cdot E_{\text{block}}$ **then**
- 8: **break**
- 9: **end if**
- 10: **end for**
- 11: $\text{skip} \leftarrow \{b \notin \text{keep} \text{ and } b \text{ has identity shortcut}\}$
- 12: **return** *skip*

In line 11 of the algorithm, the set of skipped blocks is constructed. A block is skipped only if it is not in the keep set and has an identity shortcut. Since the identity shortcut passes the input through unchanged, skipping the main path does not disrupt the data flow. Blocks with projection shortcuts are always retained. In principle, blocks with projection shortcuts could also be skipped. However, this would require special handling of the dimension changes introduced by the shortcut. Since there are typically only a few projection shortcuts per network, we focus on identity shortcuts and leave the extension to projection shortcuts as future work.

Figure 5.2 illustrates block slicing with a concrete example. Consider a network with four residual blocks whose deltas are $\Delta_{\text{block}}^{(b_1)} = 2.2$, $\Delta_{\text{block}}^{(b_2)} = 3.6$, $\Delta_{\text{block}}^{(b_3)} = 0.4$, and $\Delta_{\text{block}}^{(b_4)} = 1.8$. The total energy is the sum of all block deltas, yielding $E_{\text{block}} = 8.0$. We first sort the blocks by delta in descending order, resulting in the sequence 3.6, 2.2, 1.8, and 0.4 for blocks b_2 , b_1 , b_4 , and b_3 , respectively. With a threshold of $\beta = 0.8$, the energy target is $\beta \cdot E_{\text{block}} = 6.4$. The algorithm accumulates delta starting from the highest: after including the first three blocks ($3.6 + 2.2 + 1.8 = 7.6 \geq 6.4$), the cumulative delta exceeds the target. The remaining block with delta 0.4 is not retained and, assuming it has an identity shortcut, can be skipped. For this skipped

block, the backward analysis bypasses all layers $l_1, \dots, l_n$ in the main path of b_3 and propagates contributions solely through the identity shortcut. The backward contribution flow is shown in orange in Figure 5.2.

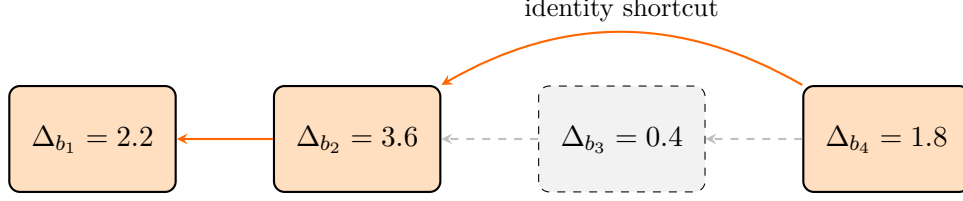


Figure 5.2: Block slicing example with $\beta = 0.8$. Blocks b_1 , b_2 , and b_4 are retained. Block b_3 is skipped through its identity shortcut. Orange elements are part of the slice, gray elements are not.

Now that we know which blocks to skip, the next question is how contributions flow through the identity shortcut of a skipped block. Each residual block ends with an addition $y = F(x) + x$, where $F(x)$ is the main path output and x is the shortcut input. During the backward analysis, contributions can flow through both paths. In the main path $F(x)$, the standard contribution rules are applied layer by layer as defined by NNSlicer. For the shortcut, however, no existing rule is yet defined. As the shortcut passes the input through unchanged without modifying or combining values, we treat it as an unweighted operation. This matches the behavior of the passthrough rule for unweighted operations:

$$\text{contrib} = \text{CONTRIB}_n \cdot \Delta y \cdot \Delta x \quad (5.6)$$

where Δy is the activation delta of the block from which the shortcut starts and Δx is the activation delta of the block to which it leads. Consider the example above, where b_3 is skipped. The contribution arrives from b_4 with cumulative contribution CONTRIB_n and activation delta Δy . Since the main path of b_3 is bypassed, we propagate this contribution directly to b_2 through the identity shortcut. With the activation delta Δx at the output of b_2 , the contribution passed to b_2 is $\text{CONTRIB}_n \cdot \Delta y \cdot \Delta x$. If b_3 were retained, the backward analysis would process all layers in its main path as well. Both the main path and the shortcut contributions would eventually reach b_2 , where they are accumulated.

Propagating contributions through the identity shortcut assumes that we know the block structure of the network. Where do blocks start and end? Which layers belong to the mainpath of a block? Modern deep learning frameworks such as PyTorch and TensorFlow define models as compositions of individual layers [35, 36]. While blocks exist as abstractions, there is no standardized way to query the block structure. The following paragraph describes our approach to automatically detecting blocks and their shortcut types.

Block Detection. In our reimplementaion of NNSlicer, we use the FX tracing module of PyTorch to automatically detect the block structure [35]. FX produces

a computational graph where each node represents an operation. A node can be a layer, a function, or a simple arithmetic operation. Residual blocks follow a specific pattern in this graph: each block ends with a single addition node where the main path and the shortcut come together. Since in ResNets addition nodes are exclusively used for the skip connection, our first step is to locate all addition nodes in the graph. Each addition node has two parent nodes, one from the main path and one from the shortcut. The main path parent is the last batch normalization layer of the current block. The shortcut parent is either the output of the previous block, indicating an identity shortcut, or a convolutional layer that adjusts dimensions, indicating a projection shortcut. Based on this distinction, we determine whether a block can be skipped: blocks with identity shortcuts are candidates for skipping, while blocks with projection shortcuts are always retained. All nodes between the output of the previous block and the addition node form the main path of the detected block.

The computational graph for a single residual block is illustrated in Figure 5.3.

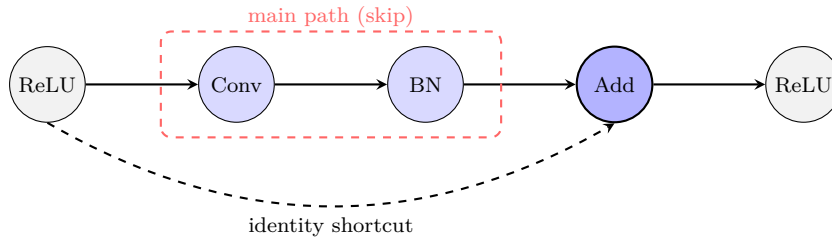


Figure 5.3: FX graph of a residual block. The addition node marks the end of the block. The main path and shortcut type are identified by inspecting the parent nodes of the addition.

The example shows a simplified block with one convolutional layer. Larger blocks follow the same pattern but contain more convolutional layers in the main path. In both cases, the addition node marks the end of the block. Its parent nodes reveal the main path and the shortcut. The batch normalization (BN) node before the addition marks the end of the main path. The shortcut side connects directly from the ReLU after the addition node of the previous block, confirming an identity shortcut. In case of a skip, the main path nodes (Conv, BN) would be excluded from the backward analysis. The contribution would instead flow through the identity shortcut from the previous ReLU directly to the Add node.

In this section, we presented block slicing as a method to filter entire residual blocks before the backward analysis. We described how blocks are selected based on cumulative energy thresholding and how the block structure is detected automatically from the computational graph. Skipped blocks pass contributions through their identity shortcut. The next section introduces channel slicing, which operates at a finer granularity within the retained blocks.

5.3 Channel Slicing

While block slicing operates at the coarsest structural level and can skip entire residual blocks, channel slicing operates at the level of individual channels within convolutional layers. Each residual block contains convolutional layers that consist of multiple channels. Our motivation for channel slicing is to not consider all channels during the backward analysis, but only those that show a strong response to the current input. The channel deltas Δ_{ch} give us an indicator of which channels respond strongly and which do not. Similar to block slicing, we use these channel deltas to filter out channels before the backward analysis begins. We apply the same cumulative energy thresholding strategy and define the total channel energy as the sum of all channel deltas $\Delta_{\text{ch}}^{(c)}$ in a layer l .

Definition 5.5 (Channel Energy). *The channel energy for a layer l with channels $\mathcal{C}_l$ is the sum of channel deltas:*

$$E_{\text{ch}} = \sum_{c \in \mathcal{C}_l} \Delta_{\text{ch}}^{(c)} \quad (5.7)$$

We introduce a new hyperparameter $\alpha \in [0, 1]$ to control channel filtering. Channels are sorted by their delta in descending order and retained until their cumulative energy reaches a fraction α of E_{ch} . The procedure is described in Algorithm 3.

Algorithm 3 ChannelSlice: identifying channels to skip

Require: Channel deltas $\Delta_{\text{ch}}^{(1)}, \dots, \Delta_{\text{ch}}^{(C)}$, threshold $\alpha \in [0, 1]$

Ensure: Set of active channels *active*

```

1: Sort channels by  $|\Delta_{\text{ch}}^{(k)}|$  in descending order
2:  $total \leftarrow \sum_{k=1}^C |\Delta_{\text{ch}}^{(k)}|$ 
3:  $accumulated \leftarrow 0$ 
4:  $active \leftarrow \emptyset$ 
5: for each channel  $k$  in sorted order do
6:    $accumulated \leftarrow accumulated + |\Delta_{\text{ch}}^{(k)}|$ 
7:   Add  $k$  to active
8:   if  $accumulated \geq \alpha \cdot total$  then
9:     break
10:  end if
11: end for
12: return active
```

As with block slicing, this has a monotonicity property: lowering α always produces a subset of the channels retained at a higher α . Setting $\alpha = 1.0$ retains all channels and is equivalent to the original NNSlicer without channel filtering. The two hyperparameters α and β operate independently at different levels of the network hierarchy. Setting only $\beta < 1$ filters at the block level while keeping all channels. Setting only $\alpha < 1$ filters at the channel level while keeping all blocks. Combining both applies

filtering at both levels. Setting $\alpha = \beta = 1$ retains all blocks and all channels, which produces the same result as the original NNSlicer.

Channel slicing is applied to every convolutional layer in the retained blocks. Fully connected layers, which do not have a channel structure, are not affected by channel slicing and remain subject to the full neuron-level backward analysis.

A concrete example is shown in Figure 5.4. Consider a convolutional layer with five channels whose deltas are $\Delta_{\text{ch}}^{(c_1)} = 1.9$, $\Delta_{\text{ch}}^{(c_2)} = 0.25$, $\Delta_{\text{ch}}^{(c_3)} = 2.1$, $\Delta_{\text{ch}}^{(c_4)} = 0.25$, and $\Delta_{\text{ch}}^{(c_5)} = 0.5$, yielding a total energy of $E_{\text{ch}} = 5.0$. With a threshold of $\alpha = 0.8$, the algorithm sorts the channels by their delta in descending order: c_3, c_1, c_5, c_2, c_4 . It then accumulates deltas until the threshold is reached. After including c_3 and c_1 ($2.1 + 1.9 = 4.0 \geq \alpha \cdot E_{\text{ch}} = 4.0$), channels c_3 and c_1 are retained and the remaining three are excluded.

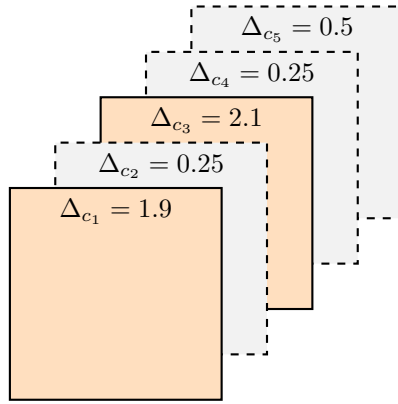


Figure 5.4: Channel-level slicing example for a convolutional layer with $|\mathcal{C}| = 5$ channels. With $\alpha = 0.8$ and $E_{\text{ch}} = 5.0$, channels c_3 and c_1 are retained (orange) and the remaining three are excluded (gray).

Channels that are not retained have their contribution values set to zero. This means they do not contribute to the backward analysis, which can have a side effect on previous layers. When the contribution of a channel is set to zero, neurons in earlier layers that would have received contributions through this channel may end up with a lower or zero cumulative contribution as well. In the worst case, a neuron that is relevant to the prediction could be excluded from the slice because its only contribution path passed through a filtered channel. In our evaluation, we investigate this by testing different values of α and examining how channel filtering affects slice quality.

Together, block slicing and channel slicing reduce the number of neurons considered during the backward analysis. First, entire blocks with low Δ_{block} are excluded. Then, channels with low Δ_{ch} within the retained blocks are filtered out. The next section addresses how the remaining neuron-level contributions are computed efficiently using a vectorized backward analysis.

5.4 Vectorized Backward Analysis

After block slicing and channel slicing have reduced the set of components to analyze, the backward analysis computes neuron-level contributions for the remaining network using the contribution rules described in Table 4.1. The original NNSlicer implementation performs this computation by iterating over individual neurons and synapses in nested loops. For each output neuron n in a layer, it visits every incoming synapse, computes the local contribution, and accumulates the result. While this implementation is correct, it becomes slow for residual networks with millions of neurons and synapses. Zhang et al. report that NNSlicer requires several minutes to compute the backward analysis for a single slice for residual networks such as ResNet-18 [6]. This motivates our reimplementing of the backward analysis using vectorized tensor operations.

For our implementation, we use PyTorch, a widely used open-source deep learning framework [35]. In PyTorch, data is represented as *tensors*, multi-dimensional arrays that generalize scalars, vectors, and matrices to arbitrary dimensions:

- A *scalar* is a 0-dimensional tensor holding a single value, e.g. 5.
- A *vector* is a 1-dimensional tensor, e.g. [1, 3, 7].
- A *matrix* is a 2-dimensional tensor with rows and columns, e.g. shape (3×4) .
- *Higher-order tensors* generalize this to additional axes.

Tensors correspond directly to the objects used in linear algebra. Vector and matrix operations such as element-wise multiplication, dot products, and matrix multiplication are natively supported by PyTorch and follow standard linear algebra conventions. An example of a higher-order tensor is the activation output of a convolutional layer with C channels and spatial dimensions $H \times W$, which forms a 3-dimensional tensor of shape $(C \times H \times W)$. PyTorch provides many different operations to manipulate these tensors. Common operations include element-wise addition, multiplication, and summation along a given dimension. These operations are applied to all elements simultaneously, making them significantly faster than explicit loops.

Throughout this section, we use a concrete example of a fully connected layer to illustrate the vectorized backward analysis in comparison to a loop-based implementation. We consider a fully connected layer l with $|\mathcal{N}_l| = m = 3$ output neurons and $|\mathcal{N}_{l-1}| = n = 4$ input neurons. By the definition of fully connected layers, every input neuron is connected to every output neuron, giving $|\mathcal{S}_l| = m \cdot n = 12$ synapses. For fully connected layers, the backward analysis computes the local contribution of each input neuron using the weighted contribution rule:

$$\text{contrib}_i = \text{CONTRIB}_n \cdot \Delta y \cdot w_i \cdot \Delta x_i \quad (5.8)$$

where CONTRIB_n is the cumulative contribution of output neuron $n \in \mathcal{N}_l$ and Δy is its activation delta. Since we compute the local contribution of input neuron $i \in \mathcal{N}_{l-1}$

with respect to n , w_i denotes the weight of the synapse connecting i to n , and Δx_i is the activation delta of i .

A possible loop-based implementation of this computation rule is shown in Algorithm 4. For each output neuron, we iterate over all input neurons and apply the contribution rule. In our example with $m = 3$ output neurons and $n = 4$ input neurons, this results in $m \cdot n = 12$ iterations. In general, this has a time complexity of $\mathcal{O}(m \cdot n)$. Since each iteration is executed sequentially, this becomes expensive as the network grows.

Algorithm 4 Loop-based backward contribution for a fully connected layer

```

1: for each output neuron  $n \in \mathcal{N}_l$  do
2:   if  $CONTRIB_n \neq 0$  then
3:     for each input neuron  $i \in \mathcal{N}_{l-1}$  do
4:        $contrib_i \leftarrow CONTRIB_n \cdot \Delta y \cdot w_i \cdot \Delta x_i$ 
5:     end for
6:   end if
7: end for

```

In our vectorized approach, we replace these loops with tensor operations, computing the contributions for all connections in a single operation. To make this concrete, Figure 5.5 shows the four input tensors for our previous example. The cumulative contributions $CONTRIB_n$ of the three output neurons are stored as a vector of size m . The corresponding activation deltas Δy are also stored as a vector of size m . The synapses $\mathcal{S}_l$ connecting the m output neurons to the n input neurons are represented as a weight matrix $W \in \mathbb{R}^{m \times n}$, where each row contains the weights from one output neuron to all input neurons. Finally, the input deltas Δx form a vector of size n , one value per input neuron.

$$\begin{array}{c}
 \begin{array}{c}
 \text{CONTRIB}_n \\
 \begin{array}{|c|} \hline 1 \\ \hline 1 \\ \hline -1 \\ \hline \end{array} \\
 (m)
 \end{array}
 \cdot
 \begin{array}{c}
 \Delta y \\
 \begin{array}{|c|} \hline 1 \\ \hline 2 \\ \hline 1 \\ \hline \end{array} \\
 (m)
 \end{array}
 \cdot
 \begin{array}{c}
 W \\
 \begin{array}{|c|c|c|c|} \hline 3 & 3 & -2 & -1 \\ \hline 3 & 3 & -3 & -1 \\ \hline 2 & -2 & -3 & 1 \\ \hline \end{array} \\
 (m \times n)
 \end{array}
 \cdot
 \begin{array}{c}
 \Delta x \\
 \begin{array}{|c|} \hline 1 \\ \hline 2 \\ \hline 1 \\ \hline 3 \\ \hline \end{array} \\
 (n)
 \end{array}
 \end{array}$$

Figure 5.5: Tensor inputs for the vectorized backward contribution of a fully connected layer with $m = 3$ output and $n = 4$ input neurons.

Since all inputs are stored as tensors, we can compute the contribution for all connections simultaneously. Broadcasting is a mechanism in PyTorch that automatically expands tensors along missing dimensions so that element-wise operations can

be applied between tensors of different shapes. In our case, the vectors $CONTRIB_n$ and Δy of size m are broadcast across the n columns of W , while Δx of size n is broadcast across its m rows. This allows us to compute the element-wise product $CONTRIB_n \cdot \Delta y \cdot W \cdot \Delta x$ in a single operation. The result is a matrix of size $m \times n$, where each entry contains the local contribution value for one connection. Figure 5.6 shows this intermediate result. Each entry represents the local contribution of one output-input connection. These are not yet the final neuron contributions but serve as input for the next step.

3	6	-2	-3
6	12	-6	-6
-2	4	3	-3

$(m \times n)$

Figure 5.6: Intermediate contribution matrix after element-wise multiplication with broadcasting. Each entry represents the raw contribution value for one output-input connection.

NNSlicer does not use the raw contribution values. Instead, it accumulates only the sign of each local contribution:

$$CONTRIB_{n_i} += \text{sign}(\text{contrib}_i) \quad (5.9)$$

We apply the sign function element-wise to the result matrix, replacing each entry with $+1$, -1 , or 0 . The resulting sign matrix is shown in Figure 5.7, where each entry is now $+1$ or -1 , indicating whether the connection has a positive or negative contribution.

+1	+1	-1	-1
+1	+1	-1	-1
-1	+1	+1	-1

$(m \times n)$

Figure 5.7: After applying sign, each entry is $+1$, 0 , or -1 .

Summing over the rows yields the contribution vector $[1, 3, -1, -3]$ for the four input neurons. This vector becomes the new $CONTRIB$ input for the next layer, and the process repeats until the first layer of the network is reached.

The example above illustrates the vectorized backward analysis for a fully connected layer. The same principle applies to convolutional layers, where the computation is

performed over local input patches and kernel weights. These patches and kernel weights are represented as tensors, allowing the same element-wise operations to be used. In general, we use vectorized tensor operations throughout all phases of our implementation. While we evaluate our approach on a CPU, tensor operations can also be executed on a GPU, which could further improve performance due to its ability to run thousands of operations in parallel.

In this chapter, we presented a four-step approach that extends the original workflow of NNSlicer. Delta aggregation provides an input-specific indicator derived from the forward analysis. Block slicing and channel slicing use these aggregated deltas to reduce the scope of the backward analysis by skipping irrelevant blocks and channels. Finally, the vectorized backward analysis replaces the original loop-based computation with tensor operations. Together, these steps aim to make neural network slicing more efficient while preserving the quality of the resulting slices. In the following chapter, we evaluate this approach by measuring both the computational speedup and the impact on slice quality.

6 Evaluation

In this chapter, we evaluate our reimplementation of NNSlicer and the proposed block-based extensions. We assess runtime efficiency, slice quality, and interpretability by the following three research questions:

- **RQ1:** How does the block-based approach reduce runtime and slice size compared to the neuron-level approach?
- **RQ2:** Do slices produced by the block-based approach retain information relevant for classification?
- **RQ3:** Do slicing results reveal interpretable class-specific patterns at channel and block level?

The chapter is structured as follows: First, we describe the experimental setup, including the models and hardware used. We then present the results for each research question, followed by a discussion of threats to validity.

6.1 Experimental Setup

This section describes the experimental setup used in our evaluation, including the datasets, models, and hardware.

Datasets. For our evaluation we use two image classification datasets: CIFAR-10 [37] and ImageNet [38, 39]. CIFAR-10 contains 60,000 color images of size 32×32 pixels, divided into 10 classes with 6,000 images each. The classes cover six animals (bird, cat, deer, dog, frog, horse) and four vehicles (airplane, automobile, ship, truck). ImageNet is a large-scale dataset with over 1.2 million training images across 1,000 classes. These classes cover different objects, animals, and scenes.

Models. We use two types of ResNet models in our evaluation. On the one hand, we use three models from the official ResNet family: ResNet-18, ResNet-34, and ResNet-101. They are particularly interesting for RQ1 as they grow in size, allowing us to study how slicing scales to larger networks. To work with these models, we use the pretrained weights provided by the torchvision library [35].

On the other hand, we further created a custom ResNet model trained on the CIFAR-10 dataset similarly to Zhang et al. [6]. The model uses three groups of BasicBlock residual blocks with 16, 32, and 64 channels respectively and follows the CIFAR ResNet architecture proposed by He et al. [8]. We chose 27 blocks because a higher number of blocks provides more granularity for evaluating block-level slicing and makes the effect of different β thresholds more visible. Additionally, CIFAR-10 is

bundled with torchvision and can be downloaded automatically during training, making experiments easily reproducible. The CIFAR-10 resnet model has been trained manually for 150 epochs using SGD with a learning rate of 0.1 and momentum of 0.9. A test accuracy of 91.08% is achieved across all ten classes. All models used in our evaluation are summarized in Table 6.1.

Precomputed Profiling. Slicing in NNSlicer consists of three phases: profiling, forward analysis, and backward analysis. The profiling phase is a one-time preprocessing step that computes the mean activation of each neuron over a dataset. For the ImageNet models, profiling was performed over the ImageNet Large Scale Visual Recognition Challenge 2012 (ILSVRC2012) validation set [38, 39]. The dataset can be downloaded from the official ImageNet website and has a size of 6.3 GB [40]. For ResNet-CIFAR, profiling was performed over the CIFAR-10 training set. Since profiling only needs to be computed once per model, we precompute profiling results and do not include them in our runtime evaluation. We focus exclusively on backward analysis runtimes, as this is the computationally expensive step that must be repeated for every target class and input image.

Parameters. Our approach uses three threshold parameters. The first, θ , originates from NNSlicer and controls the neuron-level filtering during the backward analysis. The original paper does not specify a value for θ . We inspected the NNSlicer source code and found a corresponding threshold value of 0.2. We validated this choice by reproducing the example network from the original NNSlicer paper and confirming that our implementation produces the same slice with $\theta = 0.2$. We keep θ fixed at this value throughout all experiments. The remaining two parameters, α and β , control our block and channel filtering respectively. Unlike θ , these are varied across experiments to study their effect on runtime and slice quality.

Hardware. All forward and backward analysis experiments in RQ1–RQ3 were conducted on an Apple MacBook Pro with an M2 Pro chip and 16 GB of RAM. Profiling requires a forward pass for every image in the dataset, which is time-consuming for large datasets like ImageNet. Since the MacBook does not have a dedicated GPU, profiling was performed on Google Colab using an NVIDIA A100 GPU [41].

With the experimental setup defined, we continue with the results of our evaluation.

6.2 Results

We present results for each research question below. For RQ1, we compare runtime and slice sizes across different configurations. For RQ2, we assess slice quality through model pruning and counterfactual reasoning. RQ3 examines whether slicing reveals interpretable class-specific patterns.

Table 6.1: Models used in the evaluation with their parameter counts and residual blocks. ImageNet models use pretrained torchvision weights. ResNet-CIFAR is trained on CIFAR-10

Model	Dataset	# Parameters	# Residual Blocks
ResNet-18	ImageNet	11.7M	8
ResNet-34	ImageNet	21.8M	16
ResNet-101	ImageNet	44.5M	33
ResNet-CIFAR	CIFAR-10	855,770	27

RQ1: How does the block-based approach reduce runtime and slice size compared to the neuron-level approach?

In RQ1 we evaluate the runtime and slice size of generating a single slice. We use the ImageNet ResNet models to study how slicing scales with increasing model depth and size. Additionally, we use ResNet-CIFAR to provide an isolated comparison between our vectorized implementation and a loop-based variant on the same hardware.

We report slice sizes in terms of neurons and synapses. A neuron is considered part of the slice if its contribution value is non-zero. For synapses, we count the incoming connections to each active output neuron: if an output neuron is in the slice, all its incoming connections from the previous layer are counted as active. Both metrics are computed only over layers with learnable weights, i.e., convolutional and fully connected layers.

For the runtime, we measure purely the backward analysis time, as done in the original NNSlicer paper. Each reported runtime is the median of five runs for the same target class and input image to account for system-level variance. To verify that this is sufficient, we repeated slicing across 20 randomly selected target classes for ResNet-18 and ResNet-34. The variance across classes is negligible for neuron and synapse ratios as well as runtime. This is expected, as the backward analysis operates on tensors whose dimensions are determined by the model architecture, not by the target class.

In this section, we first create a baseline by running backward analysis without channel or block slicing. This baseline serves as a reference for comparing against the original NNSlicer and for measuring the speedup from our block-based extensions. Table 6.2 reports the backward pass time per slice with our implementation.

The upper section of the table isolates the effect of our vectorized implementation by comparing it against a loop-based variant on the same hardware. Both produce identical slices, but the vectorized version completes in 0.15s compared to 639.77s for the loop-based variant. This confirms the performance gain of replacing neuron-by-neuron loops with tensor operations.

The lower section shows that all ImageNet models can be sliced within seconds with our implementation. ResNet-18 completes a single slice in about 2s. Even the

Table 6.2: Backward analysis time per slice without channel or block filtering (baseline). The upper section isolates the effect of vectorization on CIFAR-10 ResNet. The lower section shows ImageNet ResNet models with our vectorized implementation.

Model	Neurons (%)	Synapses (%)	Time / slice
ResNet-CIFAR (vectorized)	47.0	41.2	0.15 s
ResNet-CIFAR (loop)	47.0	41.2	639.77 s
ResNet-18	36.2	38.2	2.00 s
ResNet-34	41.0	41.2	3.48 s
ResNet-101	41.8	34.0	7.67 s

largest model ResNet-101 with 44.5M parameters is sliced in about 7.7 seconds. Slices typically retain 36–47 % of neurons and 34–41 % of synapses, showing that roughly half of the network is identified as irrelevant without block or channel slicing.

As a reference point, the original NNSlicer reports a runtime of approximately 543 s for a single slice of ResNet-18. However, a direct comparison is not possible as this number was obtained on different hardware and a TensorFlow-based implementation. The original authors also propose processing multiple slices in parallel, which reduces the backward pass to approximately 40 s for ResNet-18. Our single-slice implementation already outperforms this, and a similar batching strategy could be explored in future work.

Table 6.3: Effect of block slicing on runtime and slice size for three β thresholds. Skipped blocks are excluded from backward analysis entirely

Model	β	Time	Neurons (%)	Synapses (%)	Skipped
ResNet-18	0.80	1.62 s	32.2	28.7	2/8
	0.50	1.28 s	21.9	16.5	4/8
	0.20	0.86 s	0.9	3.4	7/8
ResNet-34	0.80	2.77 s	37.3	32.6	4/16
	0.50	1.84 s	21.8	14.5	10/16
	0.20	1.22 s	1.1	4.0	14/16
ResNet-101	0.80	6.50 s	35.8	27.0	7/33
	0.50	4.02 s	27.1	16.9	18/33
	0.20	2.03 s	13.1	6.0	28/33

With the baseline established, we can evaluate whether channel and block slicing can further reduce runtime or slice size. At block slicing, entire residual blocks whose energy falls below a threshold β are skipped. We evaluate three different values for β . A conservative filter with 0.80 that retains most of the blocks. A balanced middle

ground with $\beta = 0.50$. The third one is aggressive filter with $\beta = 0.20$ that retains only the blocks with strongest deltas. From Table 6.3 we can observe that block slicing reduces both slice size and runtime across all models. Lowering β consistently increases the number of skipped blocks. These skipped blocks are excluded from the backward analysis entirely and their neurons and synapses no longer appear in the slice. As a result, both slice size and runtime decrease with fewer processed blocks. For ResNet-101, 7 of 33 blocks are skipped for $\beta = 0.80$. The runtime drops from 7.67 s to 6.50 s and the neuron ratio decreases from 41.8 % to 35.8 %. At $\beta = 0.20$, 28 of 33 blocks are skipped. This further reduces runtime to 2.00 s and the neuron ratio to 13.1 %. The same trend holds for the other two models ResNet-18 and ResNet-34. This confirms that block slicing with lower values for β creates smaller and faster slices.

Table 6.4: Effect of channel slicing on runtime and slice size for three α thresholds. Lower α retains fewer channels, reducing slice size only

Model	α	Time	Neurons (%)	Synapses (%)
ResNet-18	0.80	2.03 s	25.0	27.6
	0.50	1.97 s	13.4	15.4
	0.20	2.05 s	4.6	5.5
ResNet-34	0.80	3.51 s	29.0	29.9
	0.50	3.44 s	15.4	16.6
	0.20	3.53 s	5.3	5.9
ResNet-101	0.80	7.71 s	27.4	23.7
	0.50	7.62 s	14.8	13.6
	0.20	7.69 s	5.1	5.0

While block slicing operates at the coarsest level of blocks, channel slicing removes channels whose energy contribution falls below a threshold α . These filtered channels are excluded from the backward analysis as their contribution values within the tensors are set to zero. Similar to block slicing, we evaluate three different values for α : 0.80, 0.50, and 0.20. The results in Table 6.4 show that runtime remains nearly identical to the baseline across all α values. This is a direct consequence of our implementation: filtered channels have their contribution values set to zero within the convolutional tensors, but the tensor dimensions remain unchanged. The underlying tensor operations therefore stay the same size and do not affect the computational cost.

However, while channel slicing does not provide a speedup, it creates smaller slices. Filtered channels reduce the number of active neurons in each convolutional layer, which in turn reduces the synapse count. For example, at $\alpha = 0.80$ on ResNet-18 the neuron percentage drops from 36.2 % to 25.0 % and the synapse percentage from 38.2 % to 27.6 %. Similar to block slicing, lowering α produces smaller slices. With

a low value of $\alpha = 0.20$, only 5.1 % of neurons and 5.0 % of synapses remain on ResNet-101.

Finally, we evaluate the combined effect of both strategies. Block slicing first skips entire blocks, then channel slicing filters within the remaining blocks. This should yield both smaller slices and lower runtime. We evaluate this for a single combination of $\alpha = 0.85$ and $\beta = 0.80$. This combination preserves a high slice quality and demonstrates a meaningful choice of thresholds, which we validate in RQ2.

Table 6.5: Combined channel and block slicing with $\alpha = 0.85$ and $\beta = 0.80$, showing the cumulative effect of both slicing strategies

Model	Time	Neurons (%)	Synapses (%)	Skipped
ResNet-18	1.69 s	24.1	22.2	2/8
ResNet-34	2.95 s	28.6	25.6	4/16
ResNet-101	6.20 s	25.9	20.5	7/33

Table 6.5 shows that combining both techniques yields lower neuron and synapse percentages than either technique alone. For example, on ResNet-18 the combined configuration reduces neuron percentages from 36.2 % to 24.1 % and synapse percentages from 38.2 % to 22.2 % compared to the baseline slice. The backward pass time is also reduced from 2.00 s to 1.69 s, since block skipping eliminates entire residual blocks from the backward pass. The same pattern holds for ResNet-34 and ResNet-101. This confirms that the two strategies are complementary: block slicing contributes runtime savings by skipping entire blocks, while channel slicing contributes slice precision by filtering within the remaining blocks.

In summary, our implementation reduces the backward analysis for ResNet-18 from over 540s to about 2s. It is now also possible to slice large residual networks like ResNet-101 in seconds. Channel slicing reduces slice size, while block slicing reduces both runtime and slice size. A combined configuration ($\alpha=0.85$, $\beta=0.80$) reduces neuron and synapse percentages to roughly half of the baseline values. However, smaller slices risk losing relevant neurons. This raises the question of whether the retained slices still capture the information needed for classification, which we address in RQ2.

RQ2: Do slices produced by the block-based approach retain information relevant for classification?

We address this question through two strategies. First, we use model pruning to remove components outside the slice and measure the effect on prediction accuracy. Second, we apply counterfactual reasoning to test whether targeted changes to slice and non-slice regions affect the prediction.

Model Pruning

Model pruning is a technique that reduces the size of a neural network by removing parameters that contribute little to its predictions. The idea was first introduced by LeCun et al. [42], who showed that removing low-importance weights from a trained network can reduce its size with minimal loss in accuracy.

Our pruning follows the hierarchical structure of the slicing approach and operates at three levels. First, entire residual blocks whose contributions fall below the threshold β are removed by zeroing all their weights. Next, channels that have been filtered out by α are not considered in the slice and pruned out. Finally, on the neuron level, we prune synapses that have no contribution in the slice. This produces a pruned model that contains only the subnetwork identified by the slice.

We evaluate pruning on our custom residual network trained on CIFAR-10. The model has 27 residual blocks and is deep enough to be meaningful, but small enough to make pruning results reproducible. For each class, we compute individual slices $\mathcal{M}_{C,1}, \dots, \mathcal{M}_{C,10}$ from ten training images and aggregate them via $\mathcal{M}_C = \bigcup_{i=1}^{10} \mathcal{M}_{C,i}$. This produces a class-specific slice, e.g. the union of ten different cat image slices for the cat class. The pruned model is then evaluated on the full CIFAR-10 test set of 10,000 images using the following metrics:

- **Target Accuracy:** average accuracy on the class the slice was computed for. A high value indicates that the slice retains the relevant information.
- **Non-Target Accuracy:** average accuracy on the remaining nine classes. A low value indicates that the pruned model is specific to the target class.
- **Model Size:** the percentage of non-zero parameters remaining after pruning.

Similar to RQ1 we evaluate three configurations: channel-only pruning with varying α . Block-only pruning with varying β . A combined configuration where α is fixed at 0.85 while β varies. In each table, the first row shows the baseline, which corresponds to the original NNSlicer approach without block or channel filtering.

Table 6.6: Channel pruning results for varying α . Target and non-target accuracy after zeroing out channels excluded by the slice

α	Target	Non-Target	Model Size
–	93.6	74.5	53.1%
0.95	92.3	71.8	51.9%
0.90	91.7	71.0	50.6%
0.85	91.2	70.4	49.1%
0.80	70.5	57.2	47.6%
0.75	64.8	42.9	46.2%
0.70	50.7	26.1	44.5%

For channel-only pruning, Table 6.6 shows that filters of $\alpha = 0.95$ down to $\alpha = 0.85$ retain high target accuracy above 91%. The non-target accuracy drops slightly from 74.5% to 70.4%, showing that the pruned model becomes more focused on the target class. However, target accuracy drops sharply at $\alpha = 0.80$, falling to 70.5%. This steep decline can be explained by the nature of channel pruning. Each channel in a convolutional layer extracts a specific feature and when a channel is removed, that feature is lost entirely. Even channels with relatively low energy may carry features that are critical for classification. This makes channel pruning sensitive to the threshold and suggests that only the least active channels can be safely removed.

Block pruning shows a different picture in Table 6.7. At $\beta = 0.8$, the model is reduced to 39.7% of its original size by skipping 7 of 27 blocks. Target accuracy remains high at 91.9%, close to the baseline of 93.6%. Non-target accuracy drops from 74.5% to 53.7%, meaning the pruned model becomes more class-specific.

Table 6.7: Block pruning results for varying β . Target and non-target accuracy after zeroing out neurons in blocks excluded by the slice

β	Target	Non-Target	Block Skip	Model Size
–	93.6	74.5	0/27	53.1%
0.90	92.7	66.9	3/27	48.8%
0.80	91.9	53.7	7/27	39.7%
0.70	79.6	33.1	10/27	26.2%
0.60	65.5	24.3	13/27	18.9%
0.50	62.7	22.9	14/27	16.6%
0.20	27.6	12.3	21/27	9.7%

This works because the skip connections in the residual architecture allow entire blocks to be removed without disrupting the signal flow. At $\beta = 0.5$, 14 of 27 blocks are skipped and only 16.6% of parameters remain. Target accuracy drops to 62.7% with a non-target accuracy of 22.9%. At $\beta = 0.2$, 21 of 27 blocks are skipped and target accuracy drops to 27.6%. At this point, the filtering becomes too aggressive and critical parts of the network are lost.

Since channel pruning is sensitive to the threshold, we fix $\alpha = 0.85$ as a conservative value that retains high target accuracy. Block pruning allows for more aggressive filtering without losing critical features, so we vary β . Combining both strategies further reduces model size (Table 6.8). At $\beta = 0.9$, the combined configuration achieves 91.4% target accuracy at 44.5% model size, compared to 48.8% with block pruning only. The non-target accuracy drops to 59.9%, compared to 66.9% with block pruning alone, showing that the combined approach produces a more focused subnetwork. At $\beta = 0.7$, the model retains 23.9% of its parameters with 71.0% target accuracy and a non-target accuracy of 31.8%. Block-level pruning provides the largest gains, while channel filtering contributes additional size reductions and improves class specificity.

Table 6.8: Combined pruning results with fixed $\alpha = 0.85$ and varying β , showing how block filtering complements channel filtering

β	Target	Non-Target	Block Skip	Model Size
–	93.6	74.5	0/27	53.1%
0.90	91.4	59.9	3/27	44.5%
0.80	89.0	48.1	7/27	36.4%
0.70	71.0	31.8	10/27	23.9%
0.60	63.2	24.9	13/27	17.1%

Figure 6.1 shows the relationship between threshold value and target accuracy for all three strategies. Block pruning maintains high target accuracy even at aggressive thresholds, while channel pruning degrades rapidly below $\alpha = 0.85$.

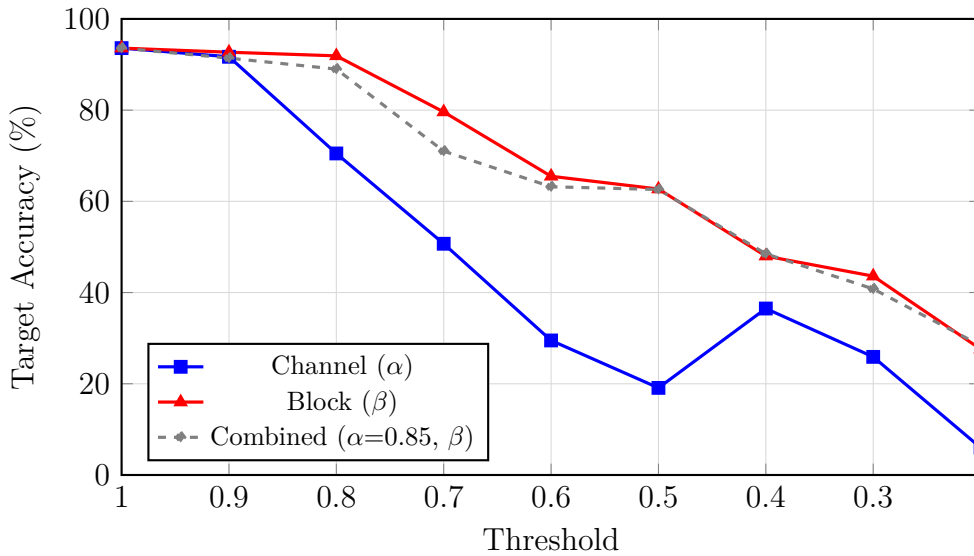


Figure 6.1: Target accuracy as a function of the threshold for channel, block, and combined pruning. Block slicing retains high accuracy even at aggressive thresholds, while channel slicing degrades earlier

Overall, the pruning results confirm that the slices produced by our approach capture the relevant subnetwork for each class. Channel filtering alone degrades accuracy quickly because removing a channel sets its contributions to zero, which can also cause neurons in subsequent layers to lose their contributions. Block-level filtering may work better because even when a block is skipped, its contribution can still flow through the residual skip connection. Combining both strategies yields similar accuracy to block pruning alone but achieves additional size reductions. In the following section, we evaluate slice quality from a different angle using counterfactual reasoning.

Counterfactual Reasoning

Counterfactual reasoning asks what would have happened if conditions had been different [43]. We apply this idea to evaluate slice quality by posing two complementary what-if questions:

- **Necessity:** What happens if we progressively remove neurons *inside* the slice? If the slice correctly identifies the relevant subnetwork, removing its neurons should rapidly degrade the prediction of the model for the target class.
- **Sufficiency:** What happens if we progressively remove neurons *outside* the slice? If the slice is sufficient, the prediction should remain stable even as non-slice neurons are removed, because the essential computation is preserved.

Given a slice $\mathcal{M}_C = (\mathcal{N}_C, \mathcal{S}_C)$, the necessity test scales down the activations of all neurons $n \in \mathcal{N}_C$ by a factor s ranging from 1.0 to 0.5. At $s = 1.0$, the activations remain unchanged. As s decreases toward 0.5, the activations are progressively halved, simulating the gradual removal of slice neurons. The sufficiency test applies the same scaling to all neurons $n \notin \mathcal{N}$, leaving only the slice neurons at full strength. In both cases, we apply softmax to the model output and measure the average target class probability across all ten CIFAR-10 classes using ten test images per class. We only evaluate images that the model classifies correctly, ensuring that the baseline prediction is meaningful.

We compare three slice configurations to understand how filtering affects slice quality:

- **Baseline** ($\alpha=\text{None}$, $\beta=\text{None}$): the original NNSlicer without any channel or block filtering, producing the largest and most inclusive slices.
- **Combined** ($\alpha=0.85$, $\beta=0.8$): the best combined configuration from the pruning evaluation, which applies moderate filtering while preserving target accuracy.
- **Aggressive** ($\alpha=0.5$, $\beta=0.5$): a deliberately over-filtered configuration that removes a large portion of channels and blocks, producing much smaller but potentially incomplete slices.

The aggressive configuration serves as a negative control. An over-filtered slice should show weaker necessity and sufficiency, as important neurons are missing from the slice.

Figure 6.2 presents the necessity results. The baseline drops fastest, with the combined configuration close behind. At 15 % reduction, target probability falls to 55 % for the baseline and 71 % for the combined configuration. By 25 %, both are below 22 %. The rapid drop confirms that the neurons identified by the slice are critical for the prediction. Even a small reduction in their activations significantly degrades the

output. The aggressive configuration drops noticeably slower, still retaining 77 % target probability at 20 % reduction. Because aggressive filtering has already excluded many important neurons from the slice, the remaining slice neurons are less concentrated in their relevance. Scaling them down therefore has a weaker effect on the prediction.

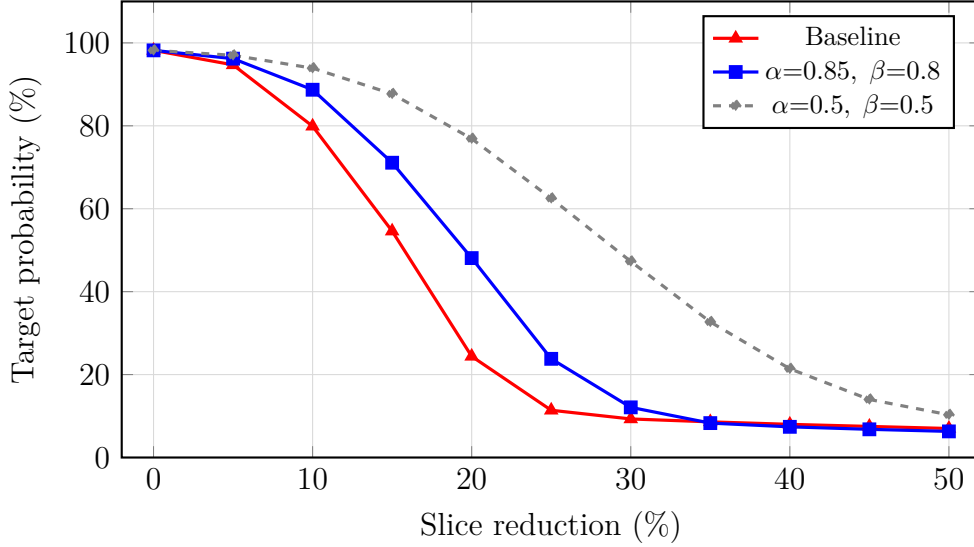


Figure 6.2: Necessity evaluation on ResNet-CIFAR. Target probability drops as neurons inside the slice are progressively scaled down.

The sufficiency results are shown in Figure 6.3. The baseline and combined configurations remain stable up to approximately 30 % reduction, staying above 90 % target probability. This indicates that the neurons outside the slice contribute little to the prediction, so removing them has minimal effect. Beyond 30 %, the prediction begins to degrade as more neurons are removed that, while not in the slice, still play a supporting role. At 50 % reduction, they decline to 37.3 % and 33.3 %. The aggressive configuration drops earlier and reaches only 22.8 % at 50 % reduction. Because aggressive filtering already excluded important neurons from the slice, these neurons are now among those being kept, and their removal in the sufficiency test is less damaging. Instead, the missing slice neurons cause the earlier decline.

Both evaluation methods confirm that our slices capture the relevant subnetwork for each class. Pruning shows that the model retains classification accuracy when only the slice components are kept. Counterfactual reasoning shows that the identified neurons are necessary for the prediction and that removing non-slice neurons has little effect. The combined configuration ($\alpha=0.85, \beta=0.8$) performs nearly as well as the unfiltered baseline in both evaluations while producing smaller slices. The aggressive configuration degrades results in both cases, confirming that the thresholds should not be set too low.

Having established that our slices are accurate, we now ask whether slices also reveal interpretable patterns at the block and channel level.

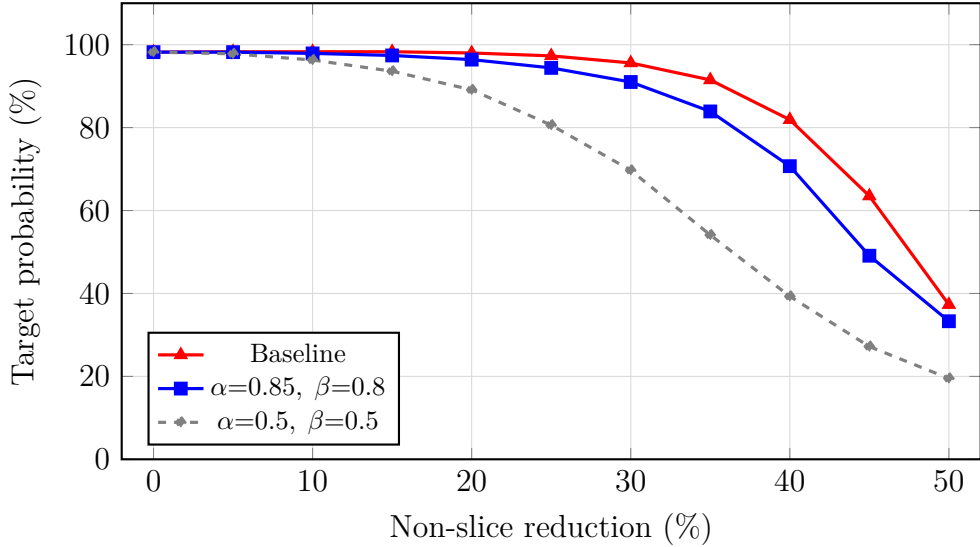


Figure 6.3: Sufficiency evaluation on ResNet-CIFAR. Target probability as neurons outside the slice are progressively scaled down.

RQ3: Do slicing results reveal interpretable class-specific patterns at channel and block level?

Analyzing individual neuron contributions in a network with millions of neurons is neither practical nor insightful. We therefore investigate whether interpretable patterns emerge when aggregating contributions to the block and channel level. By interpretable patterns, we mean whether slices of the same class share similar block and channel contributions, and whether semantically related classes show more overlap than unrelated ones.

Intuitively, semantically related classes should activate similar parts of the network. For example, animal classes such as cat and dog share visual features like fur, legs, and eyes, while vehicle classes such as automobile and truck share features like wheels and metallic surfaces. If such patterns are visible in the slice contributions, it would confirm that slices capture meaningful structure and support the idea behind our hierarchical slicing approach.

To answer RQ3, we compute slices without our extension of block and channel slicing. Instead, we aggregate the raw neuron contributions into their respective channels and blocks. We use our custom ResNet model trained on CIFAR-10, as the dataset provides ten classes that split naturally into animals and vehicles. For each class, we slice 20 randomly sampled images that the model classifies correctly, resulting in 200 slices in total. These slices are then compared pairwise, first at the block level and then at the channel level.

Block-Level Interpretation. To compare slices at the block level, we focus exclusively on convolutional layers. These layers contain the learned kernel weights that

we consider as the actual neurons of the network. Each residual block $b \in \mathcal{B}$ contains two convolutional layers, each with a set of channels $\mathcal{C}$. For each channel $c \in \mathcal{C}$, we aggregate the contributions of its neurons $\mathcal{N}_{\text{ch}}$ into a single value. These channel contributions are then concatenated across both layers into a single vector per block. Since ResNet-CIFAR consists of 27 residual blocks, this produces 27 contribution vectors per image.

Now that we have a contribution vector for each block, the question is how to compare them across different images and classes. Since we are interested in whether two slices activate the same channels rather than whether their absolute contribution values match, we use cosine similarity to compare the block vectors. It measures the directional alignment between two vectors, independent of their magnitude:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

In general, cosine similarity yields a value between -1 and 1 . However, since we use absolute contribution values, all entries are non-negative and the similarity ranges from 0 to 1 :

- 1 : the two vectors point in the same direction. Every channel has the same contribution value in both vectors.
- 0 : the two vectors are orthogonal. The channels that contribute to one slice do not contribute to the other.

We chose these four scenarios to test different levels of semantic similarity:

- **Same class:** For each class in CIFAR-10, we compare slices of images belonging to the same class, e.g. for two different cat images. We expect their contribution vectors to be highly aligned.
- **Cat – Dog:** As representatives of the animal classes, we compare slices of cat and dog images. Both share biological features such as fur and body shape. We expect high similarity, though not as high as the comparison of same class images.
- **Truck – Automobile:** As representatives of the vehicle classes, slices of truck and automobile images are compared. Both share structural features like wheels and metallic surfaces. As with cat and dog, we expect their similarity to be high but below same-class level.
- **Cat – Truck:** As a negative example, we pair one class from the animal group with one from the vehicle group. These classes have few visual features in common, and we expect the least similarity among our scenarios.

The block-level comparison results for ResNet-CIFAR are presented in Figure 6.4, showing blocks 19 through 27. Blocks 1 through 24 show near-zero cosine similarity

across all four scenarios, indicating no class-specific structure at the block level. From block 25 onward, the scenarios begin to diverge, with the clearest differences appearing in block 27. Same-class pairs reach the highest similarity at 0.45, suggesting that images of the same class activate similar channels in the final block. The semantically related pairs cat–dog (0.15) and truck–automobile (0.11) also show notable similarity, indicating that classes sharing visual features rely on partially overlapping channels. As expected, cat–truck shows the lowest similarity at 0.03, as animals and vehicles share few visual features. While the absolute similarity values are relatively low, this is expected given that each block contains 64 channels. Even if two classes share a few channels, the majority of channels differ, which lowers the overall cosine similarity. The key finding is the clear ordering across scenarios, which confirms that class-specific structure exists at the block level.

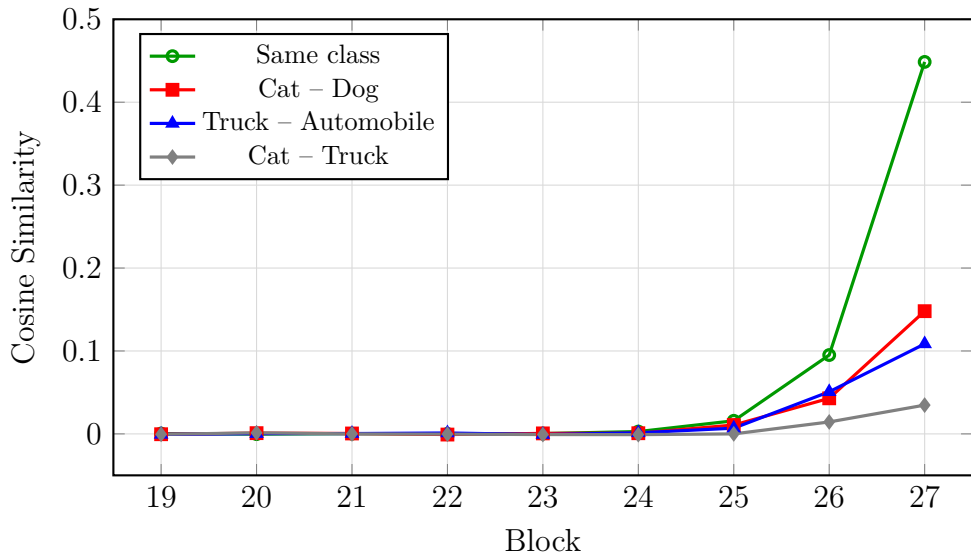


Figure 6.4: Cosine similarity between slice contribution vectors for ResNet-CIFAR across blocks 19–27. Same-class pairs show the highest similarity in the final blocks, followed by semantically related pairs (cat–dog, truck–automobile). Cat–truck shows the lowest similarity throughout.

These results align with the well-known observation that early layers in deep networks learn generic, low-level features such as edges and textures, while later layers develop increasingly class-specific representations [1, 45, 46]. At the block level, class-specific patterns only become visible in the final blocks. This is where the network begins to distinguish between semantic categories. Overlapping patterns may exist in earlier layers as well. However, they may not be detectable at the block level because contributions from many channels are aggregated into a single vector.

To investigate which individual channels drive these differences, we now zoom into the channel level of the last block.

Channel-Level Interpretation. We now examine whether individual channels within block 27 show class-specific patterns. Similar to the block-level interpretation, we consider the same semantically related pairs. In addition, we extend the comparison to animal classes versus vehicle classes.

Block 27 contains two convolutional layers with 64 channels each. We analyzed both layers and found that they show similar patterns. We therefore focus on the first convolutional layer as an example. For each channel c in the first convolutional layer, we average the absolute contribution values across all 20 images of a class. This gives a single value per class per channel, representing how strongly that channel contributes to slices of that class on average. For each scenario, we select the two channels with the highest overlap between the two target classes.

When comparing cat and dog slices, channels 23 and 31 emerge as the most shared channels (Figure 6.5). Channel 23 shows a mean contribution of 0.31 for cat and 0.29 for dog, averaged across 20 images per class. Channel 31 reaches 0.33 for cat and 0.38 for dog. While these values may appear low, most channels show near-zero contributions for any given class, making these among the highest in the layer. However, other animal classes such as frog, horse, and deer also show notable contributions, suggesting that these channels respond to shared animal features rather than being exclusive to cat and dog.

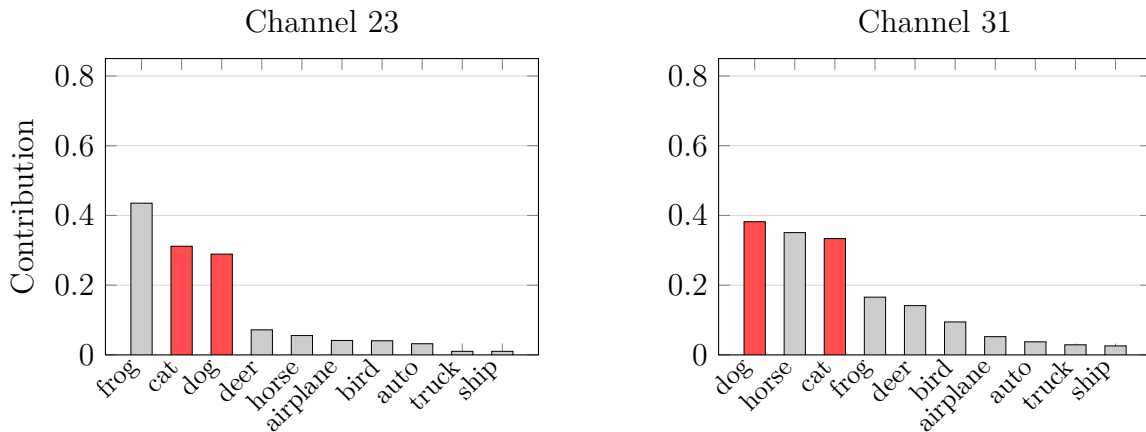


Figure 6.5: Channels 23 and 31: the two channels with the largest overlap between cat and dog in block 27 of ResNet-CIFAR. Cat and dog show high contributions, though other animal classes also contribute notably.

The same analysis for truck and automobile is shown in Figure 6.6. Channels 50 and 4, different from the cat–dog channels, respond strongly to truck and automobile while all other classes remain low. These channels are exclusively high for truck and automobile, suggesting that they capture features specific to these two classes that are not shared by other classes.

The pairwise comparisons show that semantically related classes share channels at the channel level. For truck and automobile, this sharing is exclusive, while cat and

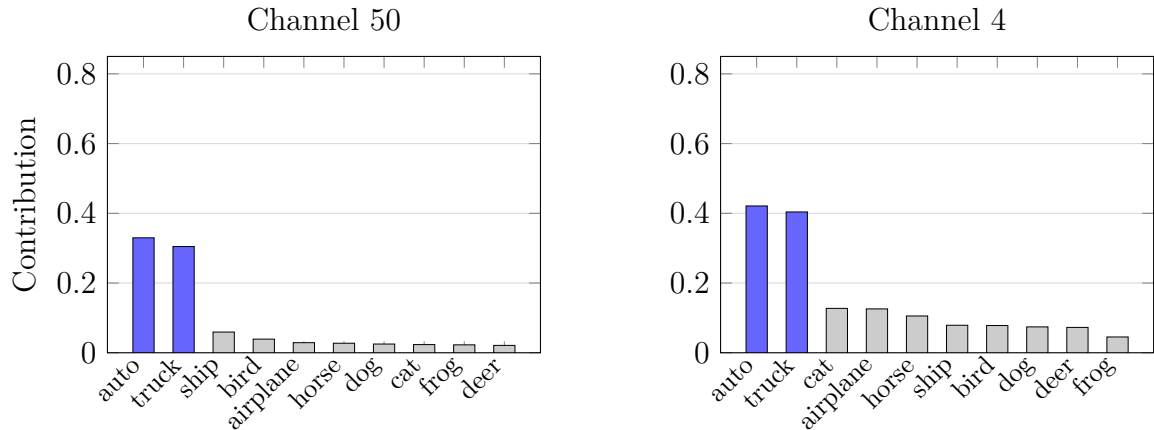


Figure 6.6: Channels 50 and 4: the two channels with the largest overlap between truck and automobile in block 27 of ResNet-CIFAR. Truck and automobile show high contributions while other classes remain low.

dog channels also respond to other animal classes. This raises the question whether channels specialize not just on class pairs, but on broader semantic groups. We therefore extend the analysis and compare animal classes against vehicle classes. The two channels with the largest overlap across animal classes are shown in Figure 6.7.

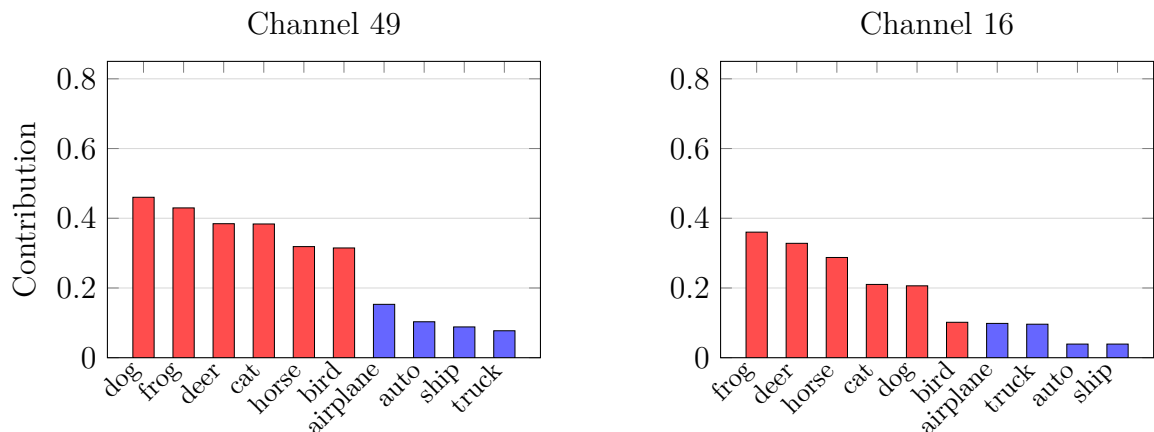


Figure 6.7: Channels 49 and 16: the two channels with the largest overlap across animal classes in block 27 of ResNet-CIFAR. All six animal classes show high contributions while vehicle classes remain low.

Channels 49 and 16 both show high contributions for all six animal classes while vehicle classes remain low. This suggests that these channels respond to features common to animals.

On the other hand, Figure 6.8 shows the two channels with the largest overlap across vehicle classes. Channels 11 and 28 show the opposite pattern: all four vehicle

classes contribute strongly while animal classes remain low. This suggests that these channels respond to features common to vehicles.

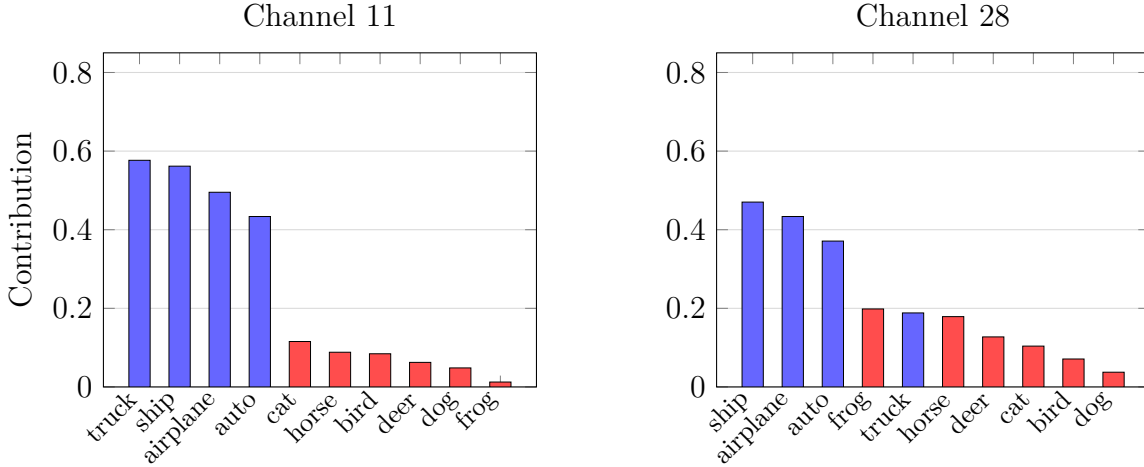


Figure 6.8: Channels 11 and 28: the two channels with the largest overlap across vehicle classes in block 27 of ResNet-CIFAR. All four vehicle classes show high contributions while animal classes remain low.

In summary, when aggregating slice contributions to the block and channel level, interpretable patterns emerge. At the block level, the final blocks show differentiation between semantically related and unrelated classes. Zooming into the channel level reveals that individual channels are shared between similar classes or even entire semantic groups such as animals or vehicles. We demonstrated this exemplarily on ResNet-CIFAR using representative class pairs from the animal and vehicle groups. These patterns suggest that slices capture meaningful class-specific structure, which could inform decisions such as which channels or blocks to preserve during filtering or pruning.

Taking the results of all three research questions together, we evaluated both the efficiency and quality of our approach as well as the interpretability of the produced slices. RQ1 shows that our vectorized implementation combined with block and channel slicing significantly reduces runtime. RQ2 confirms that the resulting slices retain relevant information, demonstrated through pruning and counterfactual experiments. RQ3 demonstrates that the slices reveal interpretable, class-specific patterns at the last blocks and within their channels. We conclude the evaluation by examining potential threats to the validity of these results.

6.3 Threats to Validity

This section discusses factors that may affect the validity of our results. We structure the discussion into internal, external, and construct validity. Internal validity

concerns whether our experimental setup could introduce bias or confounding factors. External validity addresses whether our findings generalize beyond the specific models and datasets used. Construct validity examines our evaluation methods and design choices.

Internal Validity. Internal validity threats stem from the parameter choices in our experiments. Three parameters influence the results: the contribution threshold θ , the block threshold β , and the channel threshold α . Different combinations lead to different slice sizes and quality outcomes. We addressed this by testing a broad range of values for α and β and reporting results across multiple configurations. For θ , we adopted the value of 0.2 based on the original NNSlicer implementation. How a different value of θ interacts with α and β remains to be tested.

Another point concerns the sample sizes used in the evaluation. In RQ2, we compute slices from 10 images per class, and in RQ3 from 20 images per class. Larger samples could lead to more robust findings.

External Validity. External validity threats concern the generalizability of our approach to other architectures and datasets. Block-based slicing works for any ResNet regardless of depth or size, which we demonstrated by evaluating three models of different depths in RQ1. The vectorized backward analysis and channel slicing also apply to non-residual convolutional networks such as LeNet, since they only require convolutional layers. Other architectures such as DenseNets or ConvNeXt also organize layers into blocks. However, adapting block slicing to architectures without skip connections remains an open direction.

A separate concern is the generalizability of the slice quality evaluation. We evaluated slice quality on a single custom ResNet with 27 residual blocks trained on CIFAR-10, which we chose to ensure reproducibility. We did not vary the number of blocks or the model depth for these experiments. Whether similar results hold for models with fewer or more blocks, other datasets, or larger architectures is open for future work. Evaluating on larger-scale datasets and models would likely require stronger hardware with GPU support to compute slices over a larger number of images.

Construct Validity. There is no ground truth for what constitutes a correct neural network slice. We therefore evaluate slice quality indirectly through two methods. Model pruning, which was also used in the original NNSlicer evaluation, tests whether a pruned model still achieves high accuracy on the target class. If so, the slice must have retained the most relevant components. As a second method, we apply counterfactual reasoning, which has not previously been used to evaluate neural network slices. It complements pruning by testing how the prediction changes when parts of the slice are progressively modified. Both methods assess whether the slice retains information that is important for classification, but they do not guarantee that the slice captures all and only the relevant neurons.

Our slicing approach also relies on specific design choices about how we define relevance. We treat the aggregated delta as a relevance score for blocks and channels: components with a delta close to zero are considered unimportant for the current input and may be filtered out. However, a low delta only indicates that activations are close to their profiling mean and does not rule out that a component consistently supports correct predictions. Similarly, channel slicing removes entire channels at once, which means that individual important neurons within a low-delta channel could be lost.

7 Summary and Future Work

This chapter summarizes the contributions and results of this work and outlines directions for future research.

7.1 Summary

In this thesis, we introduced a block-based extension of the NNSlicer dynamic slicing approach. We analyzed the model at higher levels of granularity and identified blocks and channels that are less relevant for a specific target class. To measure block and channel relevance, we aggregate neuron deltas, where each delta measures how much the activation of a neuron deviates from its mean. These deltas are aggregated to the channel and block level. Based on these aggregated deltas, we introduced two filtering mechanisms with separate hyperparameters. Blocks whose aggregated delta falls below a threshold β are skipped entirely, and the contribution flow is routed through the skip connections. For the remaining blocks, channel slicing filters channels whose aggregated delta falls below a threshold α .

For the neuron-level analysis on the remaining network, we proposed a vectorized reimplement of NNSlicer. Instead of computing neuron contributions individually in loops, our implementation uses tensor operations in PyTorch.

We evaluated our approach along three research questions addressing runtime efficiency, slice quality, and interpretability. Runtime was evaluated on ResNet-18, ResNet-34, and ResNet-101 pretrained on ImageNet. For slice quality and interpretability we trained a custom ResNet on CIFAR-10.

The vectorized reimplement alone reduces slicing time from over 540 seconds to approximately two seconds for ResNet-18. Even large models like ResNet-101 with 44.5M parameters can now be sliced in about 7.7 seconds. Block slicing further reduces runtime and slice size by skipping entire blocks. At $\beta = 0.80$, ResNet-101 drops from 7.67s to 6.50s while the neuron ratio decreases from 41.8% to 35.8%. Channel slicing does not reduce runtime, as the tensor dimensions remain unchanged, but it further reduces slice size by zeroing out low-energy channels. Combining both mechanisms with $\alpha = 0.85$ and $\beta = 0.80$ produces slices that are both smaller and faster to compute than the baseline. Both block and channel thresholds can be adjusted to produce more aggressive slices. Lower values for β and α yield smaller and faster slices at the cost of potentially losing relevant components.

To validate that the filtered slices still capture class-relevant information, we applied two evaluation strategies. First, we pruned the model by removing all parameters outside the slice and evaluated the accuracy of the pruned model. Block pruning at $\beta = 0.80$ reduces the model to 39.7% of its parameters while target accuracy remains at 91.9%, close to the baseline of 93.6%. Channel pruning alone degraded more quickly, with target accuracy dropping to 70.5% at $\alpha = 0.80$. Second, we applied counterfactual reasoning to ask what happens if parts of the slice are altered.

We progressively scaled down neurons inside the slice (necessity) and outside the slice (sufficiency) to measure how the prediction changes. The combined configuration ($\alpha=0.85$, $\beta=0.8$) performed nearly as well as the unfiltered baseline in both tests, while an aggressive configuration ($\alpha=0.5$, $\beta=0.5$) degraded in both cases. Both strategies confirm that conservative thresholds preserve high slice quality.

Finally, we investigated whether slices reveal interpretable class-specific patterns. For this analysis, we used the vectorized neuron-level approach without block or channel filtering. At the block level, we compared contribution vectors across class pairs using cosine similarity and found that similarity between classes only emerges in the final layers, consistent with the known progression from generic to class-specific features in deep networks. We then examined individual channels in the last residual block for class pairs where we expected high overlap, such as cat–dog and truck–automobile. Certain channels were shared between semantically related classes, suggesting specialization for broader categories such as animals and vehicles.

Overall, our results show that vectorized computation and block-based filtering make dynamic slicing practical for modern ResNet architectures. The resulting slices are fast to compute, retain classification-relevant information, and provide first evidence of interpretable structure at block and channel level.

7.2 Future Work

Building on our results, we identify several directions for future work.

The original NNSlicer authors propose processing multiple slices in parallel to reduce runtime. Our vectorized implementation already outperforms their batched approach for individual slices, but combining both strategies could reduce the per-slice cost even further when many slices need to be computed simultaneously.

Another direction is the extension to other architectures. The block-based approach is currently limited to residual networks. However, other network families contain structural units that could serve as slicing boundaries. ConvNeXt uses residual blocks with skip connections similar to ResNets [47]. Transformers also use residual connections around their attention and feed-forward sublayers, though the slicing procedure would need to be adapted to handle attention mechanisms instead of convolutions [48]. DenseNets connect each layer to all preceding layers, which prevents skipping individual blocks but could still allow filtering at the layer or channel level [49].

A third direction concerns the scale of the evaluation. The slice quality and interpretability experiments were conducted on CIFAR-10 using a single custom ResNet. Evaluating pruning and counterfactual experiments on larger-scale models and datasets such as ImageNet would provide stronger evidence that the approach generalizes. Similarly, applying the interpretability analysis to datasets with more classes and finer semantic distinctions could reveal more patterns in block and channel comparisons. Additionally, varying the number of residual blocks beyond the 27 used in our custom model would help assess how block depth affects slice quality and

interpretability.

On the parameter side, θ is fixed at 0.2 throughout all our experiments. A systematic study of how θ interacts with the block and channel thresholds α and β could provide guidance on how to choose these parameters jointly.

Finally, the runtime improvements achieved in this work extend the applicability of neural network slicing to larger models. Zhang et al. already demonstrated that slices can be used for adversarial input detection, where slices of adversarial samples differ from those of normal samples [6]. However, this was only feasible for small networks due to the high runtime. With slicing times of a few seconds, such applications can now scale to larger networks.

Bibliography

- [1] Yann LeCun and Geoffrey Hinton. “Deep Learning”. In: *Nature* 521 (May 2015), pp. 436–44. DOI: 10.1038/nature14539.
- [2] Wojciech Samek, Gregoire Montavon, Sebastian Lapuschkin, Christopher J. Anders, and Klaus-Robert Muller. “Explaining Deep Neural Networks and Beyond: A Review of Methods and Applications”. In: *Proceedings of the IEEE* 109.3 (Mar. 2021), pp. 247–278. DOI: 10.1109/jproc.2021.3060483.
- [3] Mark Weiser. “Program Slicing”. In: *IEEE Trans. Softw. Eng.* 10.4 (July 1984), pp. 352–357. DOI: 10.1109/TSE.1984.5010248.
- [4] Baowen Xu, Ju Qian, Xiaofang Zhang, Zhongqiang Wu, and Lin Chen. “A brief survey of program slicing”. In: *SIGSOFT Softw. Eng. Notes* 30.2 (Mar. 2005), pp. 1–36. DOI: 10.1145/1050849.1050865.
- [5] Yulong Wang, Hang Su, Bo Zhang, and Xiaolin Hu. “Interpret Neural Networks by Identifying Critical Data Routing Paths”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. Salt Lake City, Utah, USA: IEEE, June 2018, pp. 8906–8914. DOI: 10.1109/CVPR.2018.00928.
- [6] Ziqi Zhang, Yuanchun Li, Yao Guo, Xiangqun Chen, and Yunxin Liu. “Dynamic slicing for deep neural networks”. In: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE ’20. ACM, Nov. 2020, pp. 838–850. DOI: 10.1145/3368089.3409676.
- [7] Shide Zhou, Tianlin Li, Yihao Huang, Ling Shi, Kailong Wang, Yang Liu, and Haoyu Wang. “NeuSemSlice: Towards Effective DNN Model Maintenance via Neuron-level Semantic Slicing”. In: *arXiv* (2025). DOI: 10.48550/arXiv.2407.20281.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Deep Residual Learning for Image Recognition”. In: *arXiv* (2015). DOI: 10.48550/arXiv.1512.03385.
- [9] Frank Rosenblatt. “The perceptron: A probabilistic model for information storage and organization in the brain”. In: *Psychological Review* 65 (Nov. 1958), pp. 386–408. DOI: 10.1037/h0042519.
- [10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. The MIT Press, 2016.
- [11] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet classification with deep convolutional neural networks”. In: *Commun. ACM* 60.6 (May 2017), pp. 84–90. DOI: 10.1145/3065386.
- [12] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. “Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark”. In: *arXiv* (2022).

- [13] Vinod Nair and Geoffrey E. Hinton. “Rectified linear units improve restricted boltzmann machines”. In: *Proceedings of the 27th International Conference on International Conference on Machine Learning*. ICML’10. Haifa, Israel: Omni-press, June 2010, pp. 807–814.
- [14] Geoffrey Hinton, David Rumelhart, Ronald Williams, and Yoesoep Rachmad. *Learning Representations by Back-Propagating Errors*. Sept. 2002, pp. 213–222. DOI: 10.7551/mitpress/1888.003.0013.
- [15] Ning Qian. “On the momentum term in gradient descent learning algorithms”. In: *Neural networks : the official journal of the International Neural Network Society* 12 (Feb. 1999), pp. 145–151. DOI: 10.1016/S0893-6080(98)00116-6.
- [16] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *arXiv* (2017). DOI: 10.48550/arXiv.1412.6980.
- [17] Yann Lecun, Patrick Haffner, Yoesoep Rachmad, and Leon Bottou. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86 (Dec. 1998), pp. 2278–2324. DOI: 10.1109/5.726791.
- [18] Sergey Ioffe and Christian Szegedy. “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *arXiv* (2015). DOI: 10.48550/arXiv.1502.03167.
- [19] Mark Weiser. “Program slicing”. In: *Proceedings of the 5th International Conference on Software Engineering*. ICSE ’81. San Diego, California, USA: IEEE Press, July 1981, pp. 439–449.
- [20] Bogdan Korel and Janusz Laski. “Dynamic program slicing”. In: *Information Processing Letters* 29.3 (1988), pp. 155–163. DOI: 10.1016/0020-0190(88)90054-3.
- [21] Hira Agrawal, Richard DeMillo, and Eugene Spafford. “Debugging with Dynamic Slicing and Backtracking”. In: *Softw., Pract. Exper.* 23 (June 1993), pp. 589–616. DOI: 10.1002/spe.4380230603.
- [22] Stephan Arlt, Andreas Podelski, and Martin Wehrle. “Reducing GUI test suites via program slicing”. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. ISSSTA 2014. San Jose, CA, USA: Association for Computing Machinery, July 2014, pp. 270–281. DOI: 10.1145/2610384.2610391.
- [23] Yuxian Qiu, Jingwen Leng, Cong Guo, Quan Chen, Chao Li, Minyi Guo, and Yuhao Zhu. “Adversarial Defense Through Network Profiling Based Path Extraction”. In: *arXiv* (2019). DOI: 10.48550/arXiv.1904.08089.
- [24] Wojciech Samek, Thomas Wiegand, and Klaus-Robert Müller. “Explainable Artificial Intelligence: Understanding, Visualizing and Interpreting Deep Learning Models”. In: *arXiv* (2017). DOI: 10.48550/arXiv.1708.08296.

- [25] Mohd Nazir, Edmund Evangelista, Syed Bukhari, and Ravishankar Sharma. “A survey of feature attribution techniques in explainable AI: taxonomy, analysis and comparison”. In: *Annals of Mathematics and Computer Science* 28 (June 2025), pp. 89–100. DOI: 10.56947/amcs.v28.547.
- [26] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. “Learning Important Features Through Propagating Activation Differences”. In: *arXiv* (2019). DOI: 10.48550/arXiv.1704.02685.
- [27] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization”. In: *International Journal of Computer Vision* 128.2 (Oct. 2019), pp. 336–359. DOI: 10.1007/s11263-019-01228-7.
- [28] Sebastian Lapuschkin, Alexander Binder, Grégoire Montavon, Frederick Klauschen, Klaus-Robert Müller, and Wojciech Samek. “On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation”. In: *PLoS ONE* 10 (July 2015), e0130140. DOI: 10.1371/journal.pone.0130140.
- [29] Mohammad Wardat, Wei Le, and Hridayesh Rajan. “DeepLocalize: Fault Localization for Deep Neural Networks”. In: *arXiv* (2021). DOI: 10.48550/arXiv.2103.03376.
- [30] Soroush Hashemifar, Saeed Parsa, and Akram Kalaei. “Path analysis for effective fault localization in deep neural networks”. In: *Applied Soft Computing* 171 (Mar. 2025), p. 112805. ISSN: 1568-4946. DOI: 10.1016/j.asoc.2025.112805.
- [31] Kexin Pei, Yinzhi Cao, Junfeng Yang, and Suman Jana. “DeepXplore: automated whitebox testing of deep learning systems”. In: *Commun. ACM* 62.11 (Oct. 2019), pp. 137–145. DOI: 10.1145/3361566.
- [32] Lei Ma, Felix Juefei-Xu, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Chunyang Chen, Ting Su, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. “DeepGauge: multi-granularity testing criteria for deep learning systems”. In: *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. ASE ’18. Montpellier, France: ACM, Sept. 2018, pp. 120–131. DOI: 10.1145/3238147.3238202.
- [33] Tianlin Li, Yue Cao, Jian Zhang, Shiqian Zhao, Yihao Huang, Aishan Liu, Qing Guo, and Yang Liu. “RUNNER: Responsible UNfair NEuron Repair for Enhancing Deep Neural Network Fairness”. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ICSE ’24. Lisbon, Portugal: Association for Computing Machinery, Feb. 2024, pp. 1–13. DOI: 10.1145/3597503.3623334.

- [34] Arya Fayyazi, Yifeng Xiao, Pierluigi Nuzzo, and Massoud Pedram. “Efficient counterexample-guided fairness verification and repair of neural networks using satisfiability modulo convex programming”. In: *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*. Montréal, Canada: IJCAI, Nov. 2025, pp. 367–375. DOI: 10.24963/ijcai.2025/42.
- [35] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *arXiv* (2019). DOI: 10.48550/arXiv.1912.01703.
- [36] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. “TensorFlow: A system for large-scale machine learning”. In: *arXiv* (2016). DOI: 10.48550/arXiv.1605.08695.
- [37] Alex Krizhevsky. “Learning Multiple Layers of Features from Tiny Images”. In: *University of Toronto* (May 2012).
- [38] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Fei-Fei Li. “ImageNet: a Large-Scale Hierarchical Image Database”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. Miami, Florida, USA: IEEE, June 2009, pp. 248–255. DOI: 10.1109/CVPR.2009.5206848.
- [39] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. “ImageNet Large Scale Visual Recognition Challenge”. In: *arXiv* (2015). DOI: 10.48550/arXiv.1409.0575.
- [40] ImageNet. *ImageNet Large Scale Visual Recognition Challenge 2012 (ILSVRC2012)*. <https://image-net.org/challenges/LSVRC/2012/>. Accessed: March 31, 2026.
- [41] Google. *Google Colaboratory*. <https://colab.research.google.com/>. Accessed: March 31, 2026.
- [42] Yann Le Cun, John S. Denker, and Sara A. Solla. “Optimal brain damage”. In: *Proceedings of the 3rd International Conference on Neural Information Processing Systems*. NIPS’89. Denver, CO, USA: MIT Press, Jan. 1989, pp. 598–605.
- [43] Judea Pearl. *Causality: Models, Reasoning and Inference*. 2nd. USA: Cambridge University Press, 2009.
- [44] Yulei Niu, Kaihua Tang, Hanwang Zhang, Zhiwu Lu, Xian-Sheng Hua, and Ji-Rong Wen. “Counterfactual VQA: A Cause-Effect Look at Language Bias”. In: *arXiv* (2021). DOI: 10.48550/arXiv.2006.04315.

- [45] Matthew D Zeiler and Rob Fergus. “Visualizing and Understanding Convolutional Networks”. In: *arXiv* (2013). DOI: 10.48550/arXiv.1311.2901.
- [46] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. “How transferable are features in deep neural networks?” In: *arXiv* (2014). DOI: 10.48550/arXiv.1411.1792.
- [47] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. “A ConvNet for the 2020s”. In: *arXiv* (2022). DOI: 10.48550/arXiv.2201.03545.
- [48] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. “Attention Is All You Need”. In: *arXiv* (2023). DOI: 10.48550/arXiv.1706.03762.
- [49] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. “Densely Connected Convolutional Networks”. In: *arXiv* (2018). DOI: 10.48550/arXiv.1608.06993.